



Projet de fin d'études

Optimiser la compression vidéo par prétraitement IA : contourner les limites des outils classiques

AUTEUR

Enzo LE BODO

Étudiant IDIA-2026

enzo.lebodo@capacites.fr

ENCADREMENT

Pierre LEBRETON

Tuteur Entreprise

pierre.lebreton@capacites.fr

Matthieu PERREIRA DA SILVA

Tuteur Académique

Bruno THEILLAC

Référent Apprentissage

Capacités SAS — Cellule IXPEL

Siège social : 1 quai de Tourville 44000 Nantes, France

Adresse postale : 16, rue des Marchandises 44200 Nantes, France

Tel : (+33) 02 72 64 88 81

www.capacites.fr

Table des matières

1. Introduction	3
1.1. Contexte	3
1.1.1. Du diffuseur jusqu'au salon	3
1.1.2. Les enjeux économiques de la VOD et liens avec la recherche	5
1.1.3. Rapport d'utilisation des outils de compression	7
1.1.4. Apprentissage et filtre	9
1.2. Problématique	12
2. Présentation de l'environnement de travail	13
2.1. L'entreprise	13
2.2. La cellule IXPEL	14
3. Organisation du projet	16
3.1. Contexte du projet	16
3.2. Planning	17
3.3. Les membres du projet	17
3.4. Estimation horaire du projet	18
3.5. Outils et méthodes de travail	19
3.5.1. Méthodes de suivi et de travail	19
3.5.2. Outils de communication et de suivi	19
3.5.3. Outils internes	20
3.5.4. Adaptation et impact des outils utilisés	20
3.5.5. La vie du laboratoire	21
3.6. Limites et adaptation	21
4. Compression et qualité vidéo : défis et solutions	23
4.1. Contexte de l'équipe et formation	23
4.2. Encodage : la réduction d'informations transmises	24
4.3. Utilisateur et compression ciblée	27
4.4. Quantité d'informations des vidéos et transfert : logique et théorie	32
4.5. Comprendre le déroulement entier	32
4.6. Évaluer le contenu vidéo	34
4.7. Les limitations pour l'apprentissage	35
4.7.1. Limites mathématiques des outils pour l'apprentissage	35
4.8. Solutions existantes	37
4.8.1. Deep learning et méthodes de remplacement	37
4.8.2. Les méthodes d'optimisation existantes	38
5. Implémentation et Méthode	41
5.1. Objectif	41
5.2. Architecture globale	41
5.3. Proxy par codage neuronal	42

5.4. Proxy par codec simplifié	43
5.4.1. Sélection du meilleur bloc : du choix dur au mélange pondéré	44
5.4.2. Zone de recherche et prédiction	45
5.4.3. Quantification et transformée	45
5.4.4. Estimation du coût des images	46
5.4.5. Essais et échec d'implémentation	47
5.4.6. Bilan de l'implémentation	48
5.5. Guide d'optimisation	48
5.6. Méthode d'évaluation	49
5.7. Environnement de test	51
5.8. Données d'entraînement	51
5.9. Limites et perspectives	52
6. Résultats et analyses	53
6.1. En tant que Proxy	53
6.2. Fidélité de reconstruction	53
6.3. Corrélation de débit	55
6.4. Synthèse : deux profils, deux usages	56
6.5. Résultats face aux métriques	57
6.6. Test visuel	61
6.7. Limites et perspectives	63
7. Conclusion	65
8. Bilan personnel	66
Glossaire	67
Références bibliographiques	70
Annexe A Étude sur les plus gros challenges du secteur VOD/Streaming	72
Annexe B Architecture du filtre neuronal	73
Remerciements	74
Résumé	75

1. Introduction

1.1. Contexte

Le secteur de la **vidéo à la demande (VOD)** a connu un essor très important, notamment avec l'arrivée de nombreuses plateformes de contenu. Contrairement à la TNT, où une seule antenne émet un signal capté par un grand nombre de foyers sans coût énergétique supplémentaire par spectateur, la **VOD** nécessite une connexion point à point. Chaque clic sur « Play » sur Netflix ou Amazon Prime génère un flux dédié depuis un serveur (souvent via un **Content Delivery Network, CDN**), augmentant fortement la consommation de bande passante et d'énergie. On comprend alors que, dans ce contexte, les algorithmes de compression visant à diminuer la taille de l'information transmise de manière optimisée, les **codecs**, deviennent donc de plus en plus importants. L'évolution de leurs performances a permis de rendre ces services accessibles à de nombreuses personnes. Mais la difficulté d'évolution des architectures rend l'adoption des nouvelles versions plus complexe, ce qui pousse souvent à l'utilisation d'outils qui ne sont pas les plus optimisés.

1.1.1. Du diffuseur jusqu'au salon

La chaîne **VOD** est un processus complexe qui transforme une scène captée en une vidéo diffusée mondialement. Cette chaîne se décompose en cinq étapes majeures que l'on retrouve aussi sous forme de schéma (Fig. 1) :

- **La source** : les caméras et microphones enregistrent la vidéo et l'audio bruts. Un appareil de capture convertit ces signaux physiques en un format numérique exploitable par un ordinateur, il se passe alors aussi un grand nombre de traitements (montage) afin d'obtenir un résultat satisfaisant.
- **Encodage** (compression de la vidéo) : c'est ici qu'intervient le premier rôle clé des algorithmes de compression (**codec**). Comme les données brutes sont trop volumineuses, l'encodeur les compresse (en utilisant des standards comme **H.264**, **H.265** ou **AV1**) pour optimiser le poids du fichier sans sacrifier la qualité. Cette étape se situe côté fournisseur, avant de transmettre la vidéo.
- **Serveur** : le serveur reçoit le flux, le traite et le convertit en plusieurs versions (transcodage) pour s'adapter à différents appareils et débits.
- **CDN (distribution)** : un réseau de serveurs répartis mondialement stocke des copies du contenu. Cela permet de diffuser le flux depuis le serveur le plus proche de l'utilisateur, réduisant ainsi la latence et les interruptions.

- **Lecteur / décodeur** : c'est l'étape finale. Le lecteur (application, navigateur) reçoit le flux compressé. Le **codec** intervient ici : le décodeur transforme les données compressées en images et sons lisibles pour l'écran de l'utilisateur. Cette étape se situe côté utilisateur.

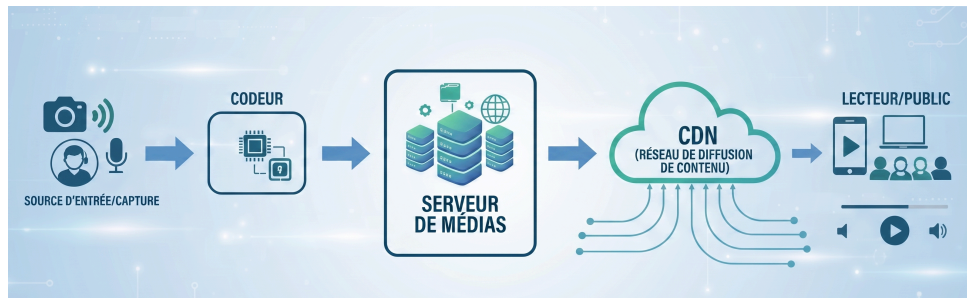


Fig. 1. – Schéma illustrant la chaîne de transmission d'une vidéo à la demande **VOD**, [1]

Ces processus concernent les données vidéo, mais aussi audio, dans notre cas, seules les données vidéo nous intéressent pour ce projet.

Il est important de mettre en avant la différence et le déséquilibre aux moments clés de l'encodage et du décodage.

Côté fournisseur, l'encodage est une opération réalisée sur les serveurs du diffuseur, ce qui, techniquement, est plus simple pour implémenter de nouvelles méthodes, car le diffuseur possède le contrôle total de son infrastructure. Bien qu'évolutive, cette infrastructure répond tout de même à une logique d'optimisation des coûts, une solution trop gourmande en calculs pourrait faire exploser les coûts pour les fournisseurs, et une solution n'apportant pas une grande optimisation mais demandant de modifier toute l'architecture serait aussi difficilement acceptable de ce point de vue.

Côté utilisateur, le décodage est effectué directement par l'appareil (TV, smartphone). Ces appareils utilisent des puces de décodage matériel conçues pour être économes en énergie et rapides. Ces puces sont figées lors de la fabrication. La seule alternative, pour un appareil dépourvu du circuit adéquat, est le décodage logiciel, c'est-à-dire faire exécuter le calcul de décodage par le processeur général. Cette voie est universelle, mais bien moins efficace. Il est donc extrêmement complexe de modifier le comportement de ce décodage ou d'y introduire de nouvelles méthodes, car cela nécessiterait de mettre à jour le matériel ou d'imposer des contraintes incompatibles avec les appareils existants. Cette limite s'explique notamment par le fait que les nouvelles solutions, apportant une optimisation, s'accompagnent d'un nombre plus important de calculs, ce qui n'est pas toujours supporté par l'ensemble des appareils.

On comprend alors que la balance se trouve, à court terme, dans des solutions d'optimisation peu gourmandes en ressources. Cela permet de répondre à l'optimisation des coûts de l'entreprise, mais aussi aux limites matérielles côté utilisateur. On peut aussi prédire des modifications futures plus importantes, advenant à la suite d'une maturité suffisante des outils nécessitant cette évolution.

De ce constat, on peut estimer qu'une optimisation réaliste se trouverait en amont du transfert de la vidéo. En particulier, car une vidéo encodée sera transmise à un grand nombre d'utilisateurs, donc optimiser une vidéo pourrait alors devenir intéressant plus rapidement.

1.1.2. Les enjeux économiques de la VOD et liens avec la recherche

Une question guide cette mise en contexte : quels sont les enjeux économiques de la **VOD**, et en quoi une optimisation en amont de la compression répond-elle aux besoins des acteurs du secteur ? Une attention sera aussi portée sur les effets d'une optimisation au regard de l'écologie, et donc comment ces enjeux d'optimisation pourraient influencer l'impact du secteur.

La **VOD** représente aujourd'hui une part majeure du trafic internet mondial, et chaque flux transmis a un coût, en bande passante, en stockage et en énergie. La vidéo reste la première catégorie d'application en volume sur les réseaux, et une poignée d'acteurs, parmi lesquels Netflix, Amazon ou Meta, concentrent à eux seuls plus de la moitié du trafic web [2].

Ce poids dans le trafic s'accompagne d'un marché en forte croissance. Le segment de la **VOD** par abonnement (SVOD, *Subscription Video On Demand*) était évalué à 151,9 milliards de dollars en 2023, et les projections l'estiment à plus de 408 milliards à l'horizon 2032, soit une croissance annuelle moyenne supérieure à 11 % [3]. Cette dynamique est un point essentiel pour notre sujet : plus le volume de contenus diffusés augmente, plus une optimisation appliquée une seule fois en amont devient rentable, puisqu'elle s'applique à l'ensemble des utilisateurs qui regarderont la vidéo. Pour des plateformes comptant des millions d'abonnés, la moindre réduction du débit moyen se traduit par des économies considérables, répétées à chaque diffusion. C'est ce qui explique l'attention portée à la compression.

Taille du marché SVOD, par contenu, 2022-2032 (milliards USD)

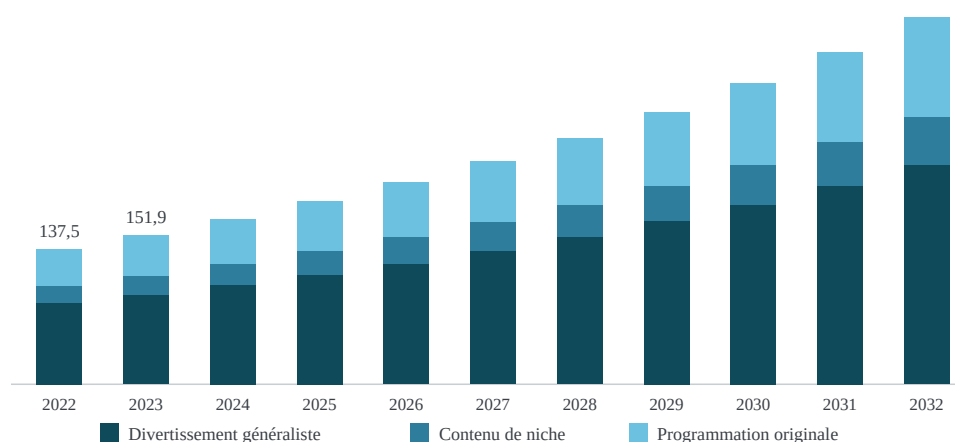


Fig. 2. – Évolution du marché SVOD par type de contenu, 2022-2032 (en milliards de dollars) [3]

Ce graphique montre la tendance du secteur, il semble important de préciser que le secteur de la vidéo ne s'arrête pas là. Ces données excluent les plateformes financées par la publicité comme YouTube, qui n'apparaît donc pas dans ce marché alors qu'elle représente à elle seule plus de 10 % de la bande passante internet mondiale [2]. Or, ces plateformes s'appuient sur des outils de compression similaires, l'enjeu des optimisations liées à la compression dépasse donc largement le seul périmètre du marché SVOD chiffré ici.

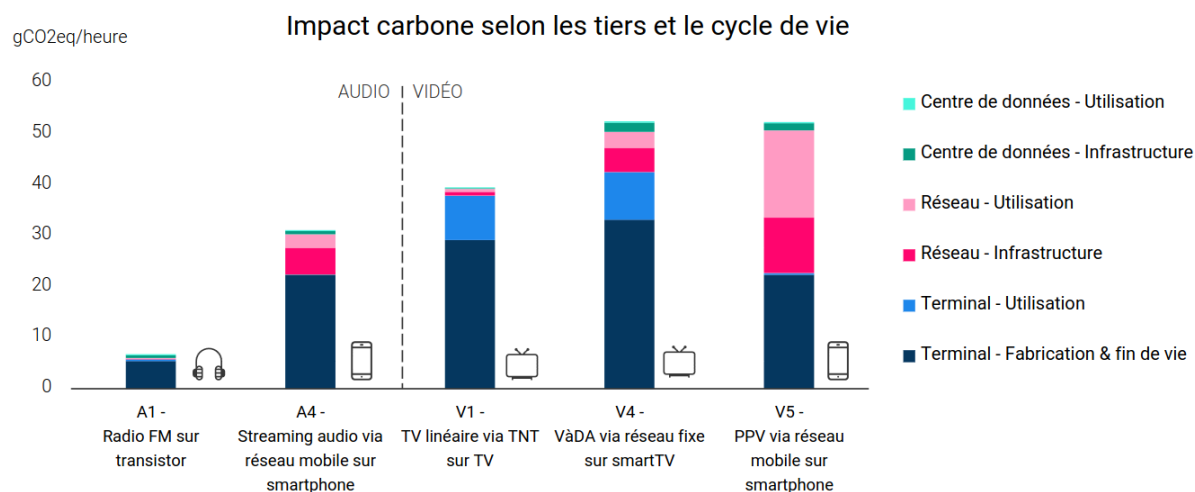


Fig. 3. – ADEME : Répartition des usages numériques et de leur impact environnemental, [4]

Ce sujet questionne aussi l'impact d'une telle optimisation sur un sujet incontournable de notre époque : l'écologie. Si l'on reprend le graphique de l'ADEME, on y retrouve de nombreuses informations liées à cette thématique, et donc des éléments de réponse. Il faut alors se référer aux sections « V4 » et « V5 », qui représentent les usages liés à la **VOD** sur télévision ou smartphone. On y remarque une importance relativement faible de ces données dans le total. Il faut aussi comprendre que ces optimisations peuvent avoir un impact direct sur les parties réseau. En fluidifiant le trafic par des vidéos moins gourmandes en ressources, on obtient des infrastructures qui supportent plus facilement la charge requise, rendant alors leur déploiement moins massif. Cet enjeu est loin d'être uniquement théorique, car le trafic est de plus en plus marqué par des pics. En 2024, les dix journées de trafic les plus importantes coïncidaient toutes avec un événement diffusé en direct, ce type d'événement pouvant faire bondir le trafic réseau de 30 à 40 % [2]. Des flux plus légers aident directement à absorber ces montées en charge.

Mais il faut aussi reprendre les éléments de la section précédente (Chapitre 1.1.1). Garder des outils de compression qui prennent en compte les limites du matériel existant rend ce matériel plus durable, utiliser des outils trop gourmands en calcul rend les traitements plus lourds et complexes, ce qui peut diminuer la durée de vie des appareils utilisés. Une adoption massive d'un codec récent et nécessitant des appareils puissants comme **AV1** entraînerait ainsi une obsolescence anticipée de certains téléviseurs actuels.

On peut en revanche opposer à cette optimisation un possible effet rebond, un accès plus simple et plus rapide à davantage de ressources **VOD** pourrait toucher plus d'utilisateurs, ou permettre de proposer du contenu de plus haute qualité. C'est précisément le but des entreprises dans une logique de performance et de qualité de service. C'est ce qu'on remarque lors des différentes évolutions, les outils deviennent plus performants mais permettent aussi d'augmenter la résolution et la qualité des contenus ce qui continue de faire croître les besoins et apporte de nouveaux besoins, comme de nouveaux téléviseurs supportant ces technologies. De ce point de vue, on irait alors à l'encontre des optimisations évoquées du point de vue écologique.

Les acteurs concernés par ce secteur sont parmi les plus gros du numérique, Netflix, Amazon ou Meta, et ce sont précisément les clients de la cellule IXPEL dans laquelle j'effectue mon alternance. Leurs besoins orientent donc directement nos sujets de recherche, la majorité de nos projets portent sur l'évaluation ou le développement de nouvelles solutions, souvent confrontées à un panel d'utilisateurs.

Les défis remontés par le secteur confortent l'intérêt d'une optimisation agissant directement sur le poids des fichiers, une étude récente place le stockage comme premier défi des entreprises de streaming interrogées (Annexe A). Réduire le poids des fichiers agit directement sur ce poste, et donc sur les coûts à différents points de la chaîne.

Enfin, un point mérite d'être souligné, l'écosystème open source et open access (libre d'accès) joue un rôle clé dans ce domaine. Il permet de faire évoluer les outils de compression, mais donne aussi accès à des outils de mesure de qualité vidéo complexes, parfois développés en interne par ces entreprises, et dont la disponibilité conditionne en grande partie la recherche dans ce domaine. Cela semble parfois en contradiction avec la volonté générale des entreprises technologiques, qui cherchent à protéger leurs outils pour ne pas aider la concurrence. Cependant, dans ce cas précis, ces entreprises profitent aussi d'une communauté très active autour des contenus vidéo. Des utilisateurs ou des groupes de recherche s'emparent de leurs outils et proposent des améliorations que leurs équipes internes ne pourraient pas toutes réaliser. Faire évoluer le secteur permet à ces entreprises d'en tirer profit. Nous l'avons vu, le secteur est en forte hausse, chaque optimisation compte et cette aide externe, gratuite, est intéressante. C'est aussi la raison de l'intérêt de ces entreprises à financer différents projets de recherche. On peut toutefois illustrer la limite de cette logique. Ces entreprises partagent peu, voire pas, leurs données, y compris celles utilisées pour produire ou entraîner des outils qui seront ensuite mis en accès libre. Cela montre que cette volonté de partage reste ciblée, éloignée d'une générosité soudaine qui ne correspondrait pas à une logique économique.

1.1.3. Rapport d'utilisation des outils de compression

Afin d'introduire les analyses présentées, qui se basent sur différents outils de compression vidéo, voici un bref historique des codecs existants.

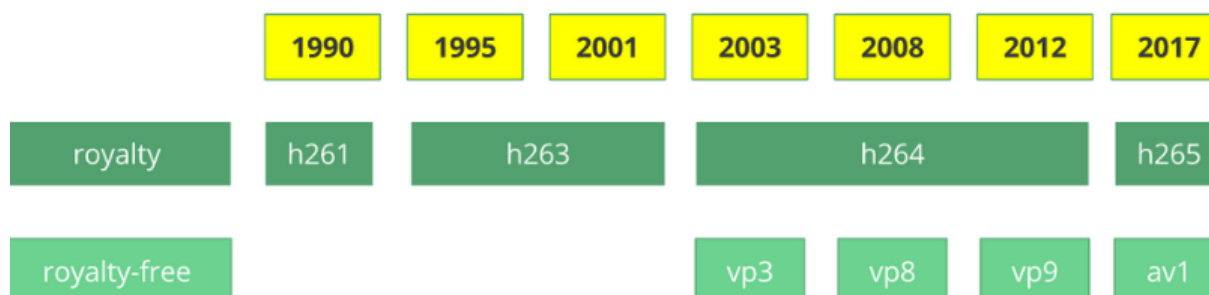


Fig. 4. – Historique et évolution des outils de compression vidéo (**codecs**) entre 1990 et 2017 [5]

On voit donc que différents outils existent, chacun possède des performances de compression différentes. On peut les regrouper en deux familles selon leur modèle de licence. D'un côté, les codecs libres de redevances (royalty-free), comme VP9 ou AV1, développés notamment par l'Alliance for Open Media, utilisables sans payer de droits. De l'autre, les codecs soumis à redevances, comme H.264 ou H.265, dont l'usage suppose l'acquisition de licences auprès des détenteurs de brevets. Ce modèle de licence pèse directement sur l'adoption. Le cas de H.265 est intéressant, sa licence, fragmentée entre plusieurs groupes de brevets, a freiné son déploiement et a précisément motivé la création d'AV1, libre de redevances. Paradoxalement, ce sont pourtant les codecs payants qui restent les plus utilisés aujourd'hui, parce qu'ils bénéficient d'un support matériel quasi universel et d'un écosystème plus mature, là où les alternatives libres, plus récentes, restent coûteuses en calcul et moins bien prises en charge par le matériel existant.

Les figures suivantes illustrent la répartition d'utilisation des outils de compression en 2023 et 2024 sur un panel d'entreprises. On peut y voir que les solutions récentes ne sont pas encore adoptées par la majorité des entreprises, cela montre bien la difficulté d'évolution.

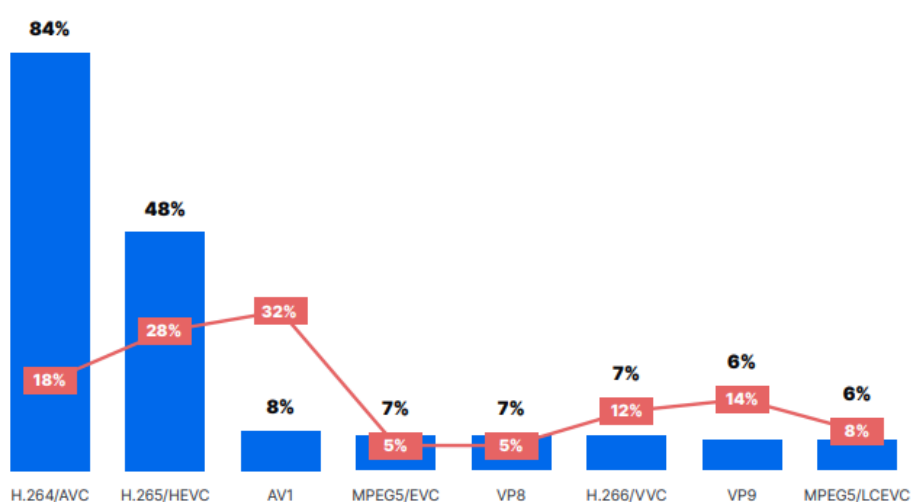


Fig. 5. – Illustration de la répartition d'utilisation des outils de compression en 2023-2024 (Streaming & VOD) pour un panel d'entreprises (en rouge : l'outil envisagé pour l'année suivante) [6]

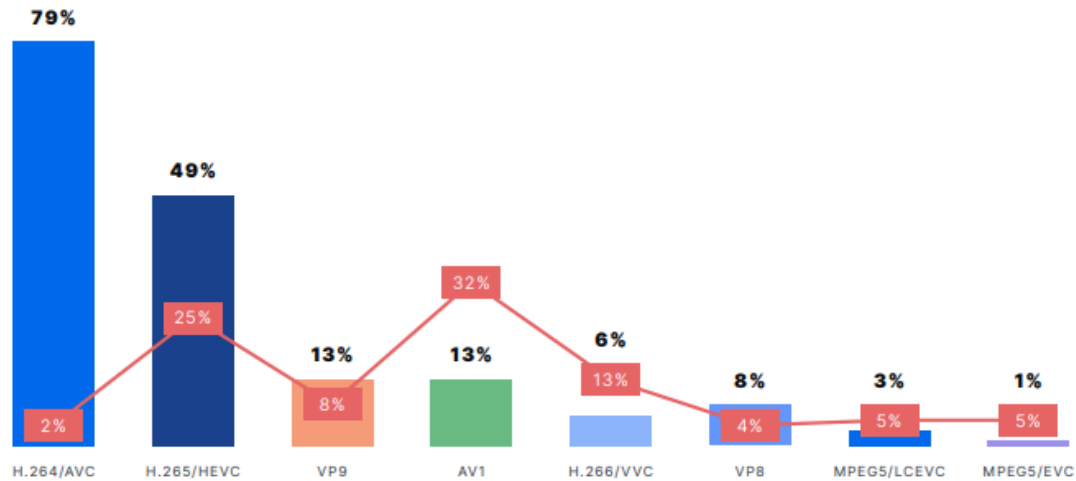


Fig. 6. – Illustration de la répartition d'utilisation des outils de compression en 2024-2025 uniquement pour la **VOD** pour un panel d'entreprises (en rouge : l'outil envisagé pour l'année suivante) [7]

Il est intéressant de noter que les chiffres évoluent peu, ce qui prouve l'écart entre la volonté d'évolution et la faisabilité réelle. On voit que le **codec** le plus utilisé en 2025 reste **H.264**, pourtant créé en 2003. Cependant, **H.265** et **AV1** représentent les candidats des prochaines années d'après ces sondages. Un point important, énoncé dans la section précédente, le matériel actuel est aussi un élément à prendre en compte. À ce jour, il existe beaucoup plus de matériel capable de décoder **H.265** nativement, ce qui le rend plus efficace et moins énergivore. Sans ce décodage natif, la tâche devient plus complexe pour des appareils plus anciens, car ils doivent se débrouiller avec la puissance des processeurs en place, ce qui, notamment avec **AV1** qui demande plus de calcul, devient alors plus compliqué, voire impossible.

Le choix d'une cible d'optimisation réaliste et qui prend en compte ces différents éléments semble donc clair : **H.265** est aujourd'hui très intéressant pour ce cas d'utilisation, pour tous les points évoqués.

1.1.4. Apprentissage et filtre

Nous avons donc vu qu'une optimisation serait intéressante économiquement. Il semble aussi assez clair que les contraintes matérielles nous poussent à orienter les solutions pour qu'elles soient au début de la boucle de transmission, c'est-à-dire avant le transfert des contenus, la partie maîtrisée par les diffuseurs.

Ce sujet va mélanger différents éléments techniques, liés à la compression vidéo mais aussi à l'apprentissage d'un réseau de neurones qui ici sera un filtre intelligent permettant de possibles optimisations en modifiant l'image qui sera encodée. Il semble donc important d'introduire certaines

notions liées à l'apprentissage de réseaux de neurones et comprendre en quoi il pourrait être intéressant pour accompagner les outils de compressions classiques.

Tout d'abord, un réseau de neurones peut se voir comme un ensemble de paramètres réglables qui se combinent pour répondre à une tâche. Il reçoit une entrée, ici une image, la fait passer à travers différentes couches de calcul, qui contiennent des « neurones ». Le but de ces étapes est de produire une sortie, dans notre cas une nouvelle image qui sera possiblement plus simple à transmettre. Chaque connexion entre deux neurones porte un poids, que l'on peut ajuster, ce sont ces poids qui déterminent le comportement d'un réseau (Fig. 7).

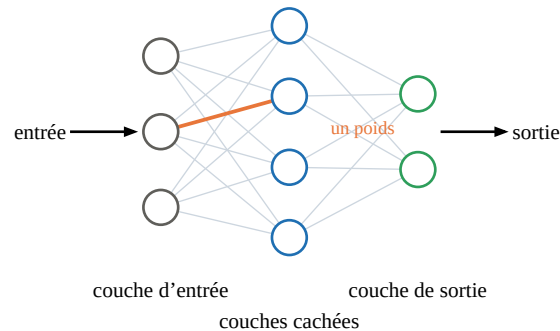


Fig. 7. – Un réseau de neurones : des couches de neurones reliés par des connexions, chacune portant un poids ajustable.

Ajuster ces poids est primordial, c'est pourquoi on parle d'apprentissage : un entraînement qui permet d'apprendre une tâche à un réseau de neurones en ajustant au mieux ces nombreux paramètres (poids).

L'entraînement consiste à montrer un grand nombre d'exemples au réseau, comparer sa sortie au résultat attendu, mesurer l'écart (l'erreur), puis ajuster légèrement les poids pour réduire cette erreur. Répétée un très grand nombre de fois, cette boucle fait converger le réseau (Fig. 8). C'est précisément ce que l'on appelle optimiser le réseau : chercher les poids qui minimisent l'erreur (convergence vers une solution optimale). L'erreur dépend du contexte d'apprentissage, il est donc important d'avoir une mesure qui suit la logique de l'objectif final, on peut voir cette mesure de l'erreur comme un guide de l'optimisation. Dans notre cas ce guide doit être le garant de la qualité d'image qui sera transmise à l'utilisateur.

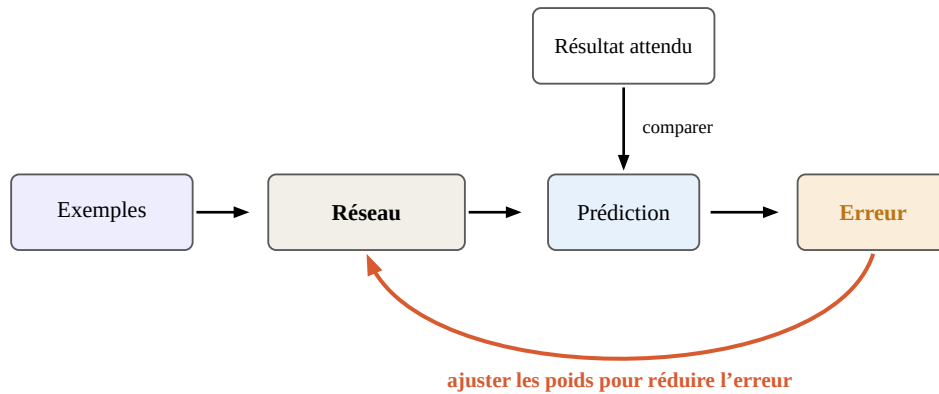


Fig. 8. – La boucle d’entraînement : le réseau prédit, on mesure l’erreur par rapport au résultat attendu, puis on ajuste les poids pour la réduire.

Dans ce projet, ce réseau prend la forme d’un filtre appliqué à l’image. On peut imaginer que deux emplacements sont possibles (Fig. 9). Un filtre de prétraitement agit sur l’image avant la compression, côté diffuseur, pour la rendre plus facile à compresser. Un filtre de post-traitement agit après la décompression, côté utilisateur, pour corriger l’image reconstruite. Les contraintes matérielles côté utilisateur rendent le post-traitement difficile à déployer. Ce projet se concentre donc sur le prétraitement. L’objectif de l’apprentissage devient alors concret : ajuster les poids du filtre pour qu’une fois l’image compressée, son poids diminue sans que l’utilisateur ne perçoive de perte de qualité.

Pour ce faire, il est important que le filtre apprenne à régler ses paramètres dans un environnement réaliste. Ce qui lui permettra de s’adapter aux limites des outils de compressions existants et donc possiblement rendre l’image plus simple à encoder.

On peut imaginer qu’un réseau de neurones soit intéressant pour ce genre d’objectif car les échanges et opérations entre neurones représentent une forme « d’intelligence », permettant de s’adapter au contenu et comprendre des représentations complexes liées aux images afin de les traiter au mieux. Par exemple, le filtre pourrait comprendre qu’une texture d’image complexe à traiter pour un codec, et donc coûteuse pour le poids total d’une vidéo, n’a que très peu d’importance concernant la qualité vidéo perçue, dans ce cas, la modifier devient très intéressant.

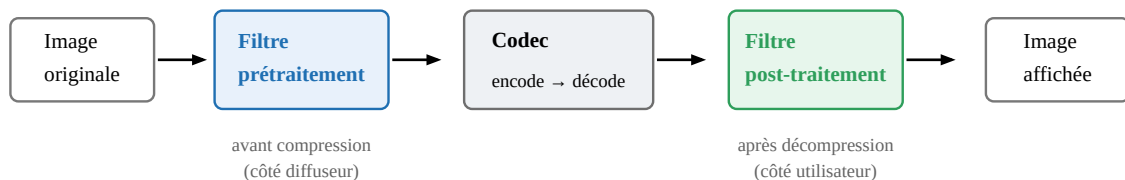


Fig. 9. – Emplacement des filtres : le prétraitement (avant la compression, côté diffuseur) et le post-traitement (après la décompression, côté utilisateur)

Derrière ces différentes étapes d’optimisation se cachent des opérations mathématiques. Nous verrons dans la suite de ce document que définir des opérations fidèles aux objectifs et utilisables dans

une boucle d'apprentissage est un défi majeur du projet. Notamment, en utilisant un environnement de simulation de la compression fiable et un moyen de mesurer la qualité de l'image (l'erreur).

1.2. Problématique

Nous avons vu que la diffusion de contenu vidéo est soumise à de fortes contraintes, en particulier matérielles côté utilisateur. Optimiser un codec déjà en place comme **H.265**, plutôt que d'en imposer un nouveau, apparaît donc comme la voie la plus réaliste, cela permet de réduire la taille des flux transmis sans toucher aux appareils des utilisateurs. C'est précisément ce que l'intelligence artificielle (IA) pourrait permettre.

L'idée explorée dans ce projet est d'utiliser l'IA en amont de la compression, pour prétraiter la vidéo. Ajuster l'image de façon à la rendre plus facile à compresser, et ainsi réduire le poids du fichier final à qualité visuelle équivalente. L'intérêt de l'IA tient à sa capacité d'adaptation. Les méthodes traditionnelles peinent parfois face à la diversité des contenus, un modèle peut apprendre à repérer ce qui compte visuellement et à modifier l'image en conséquence. On obtient une approche potentiellement plus souple, et moins coûteuse en calcul que des optimisations classiques poussées. Reste un obstacle majeur. Les codecs vidéo n'ont jamais été pensés pour servir dans un apprentissage, ils s'intègrent donc mal pour l'optimisation d'une IA. La question principale est alors de définir les moyens permettant à l'IA d'apprendre efficacement à travers un codec classique comme **H.265**, malgré les limitations de ce dernier. C'est le verrou technique central du projet.

À cette question s'en ajoutent deux autres, propres au sujet. D'abord, comment évaluer la qualité d'une vidéo optimisée par IA, alors que les outils existants sont conçus pour juger une vidéo encodée de façon classique ? Ensuite, comment guider l'apprentissage pour qu'il simule la satisfaction réelle d'un utilisateur final ? C'est un point clé, mais un défi de taille, là encore parce que les outils actuels ne sont pas faits pour cela, la plupart des projets IA et vidéo reposent sur des mesures simples, efficaces pour orienter l'apprentissage, mais aveugles aux spécificités de la perception visuelle humaine.

En résumé, ce projet questionne la faisabilité d'une optimisation de la compression vidéo par IA dans le cadre des codecs existants comme **H.265**. Il s'agit d'y intégrer l'IA pour gagner en performance tout en respectant les contraintes matérielles et les exigences de qualité visuelle. Le dépassement des verrous liés aux algorithmes de codage classiques et la définition des bons outils d'évaluation seront donc étudiés en parallèle.

Au-delà de ce verrou technique, ce rapport examine également le projet sous trois angles complémentaires : économique, organisationnel et humain.

2. Présentation de l'environnement de travail

2.1. L'entreprise

Capacités SAS est une filiale privée de valorisation de la recherche de Nantes Université. Créée en 2005, elle emploie aujourd'hui environ une centaine de collaborateurs. L'entreprise est détenue à 93 % par Nantes Université et à 7 % par la chambre de Commerce et d'Industrie de Nantes Saint-Nazaire. Elle est présente sur trois villes du Grand Ouest : La Roche-sur-Yon, Saint-Nazaire et Nantes.

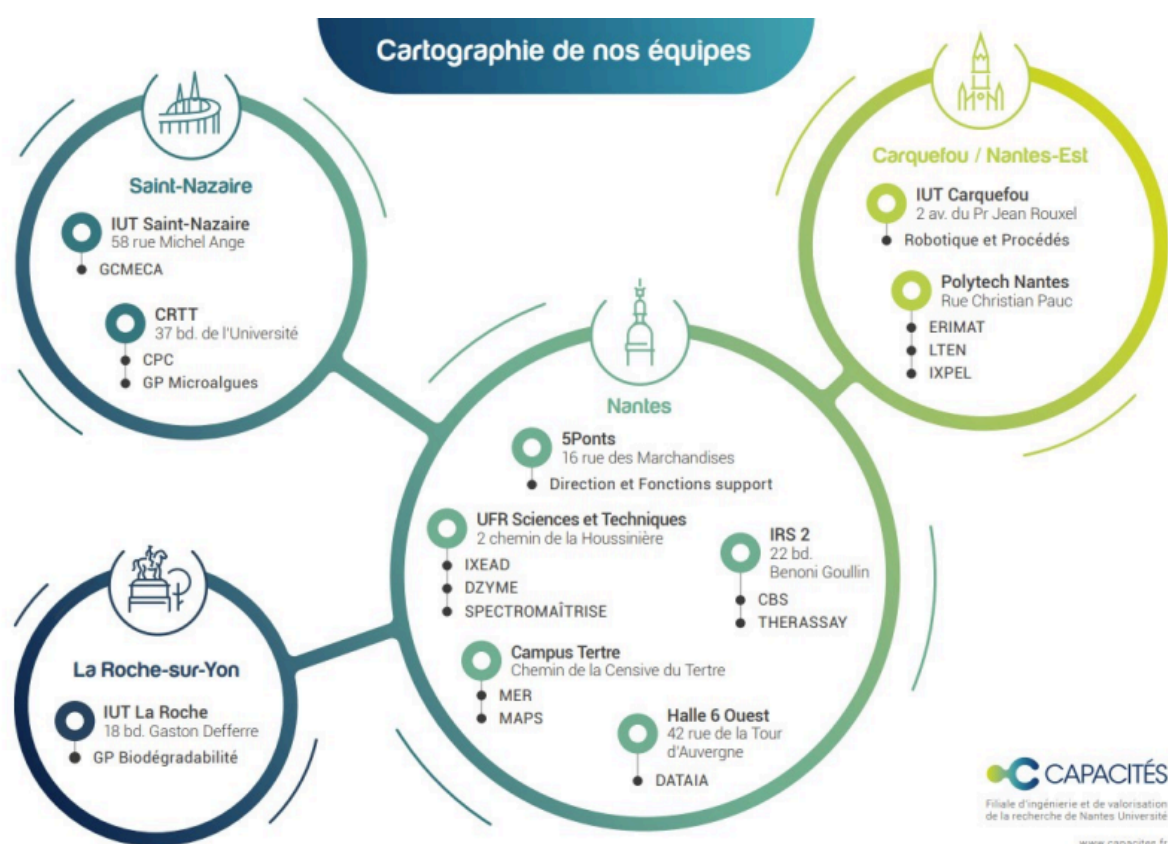


Fig. 10. – Cartographie des cellules de Capacités

Capacités est divisée en 13 cellules d'expertise. Ces cellules portent des projets d'entreprise au sein des laboratoires auxquels elles sont rattachées, afin d'avoir accès à leur expertise et de se trouver au plus près de la découverte scientifique.

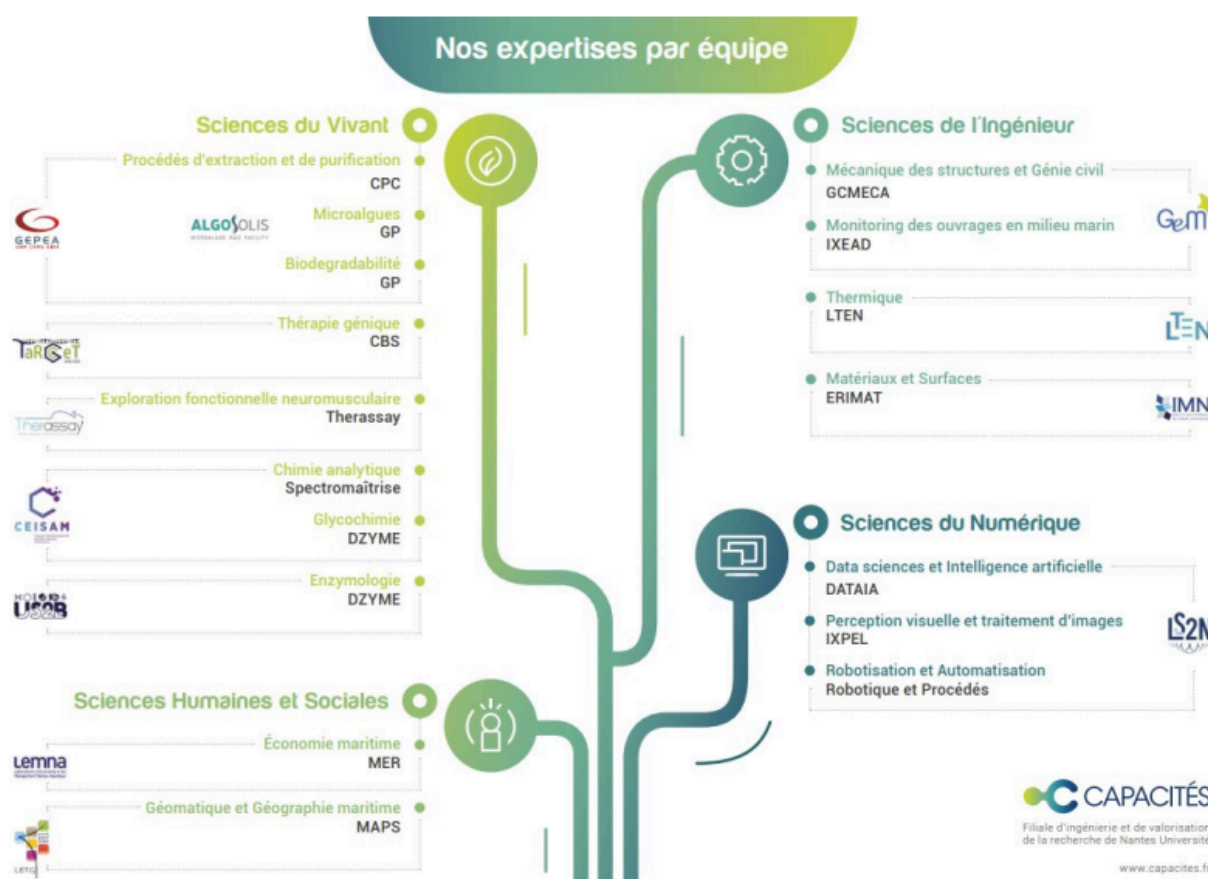


Fig. 11. – Expertises des cellules de Capacités

Elles apportent aussi un support à la recherche lors d'un besoin en ingénierie. Les cellules sont composées de chercheurs, ingénieurs et techniciens. Chaque équipe possède ses propres clients et gère une partie de son budget pour l'attribuer selon les ressources nécessaires. Il existe également des projets inter-cellules pour regrouper plusieurs domaines d'expertise sur les sujets pluridisciplinaires.

2.2. La cellule IXPEL

Je travaille au sein de la cellule IXPEL, intégrée à l'équipe de recherche IPI (Image Perception Interaction), qui appartient au LS2N (Laboratoire des Sciences du Numérique de Nantes). L'équipe est spécialisée dans l'intelligence artificielle appliquée à l'image et la qualité d'expérience. On y retrouve par exemple des sujets liés à l'imagerie médicale, au traitement de documents manuscrits et à l'expérience et qualité utilisateur face à du contenu vidéo, l'équipe est reconnue mondialement sur ce dernier sujet, ce qui lui permet de travailler en collaboration avec les plus grandes entreprises du secteur et en particulier avec leurs équipes de recherche.

Les clients de notre cellule sont de grandes entreprises du numérique comme Meta, Netflix ou Amazon. L'équipe IPI et la cellule IXPEL sont reconnues pour les tests subjectifs et la qualité d'expérience, c'est en partie sur ces thématiques que les projets en lien avec ces entreprises portent.

Les tests permettent par exemple de recueillir des données sur la satisfaction d'utilisateurs face à des contenus vidéo, ce qui permet par la suite d'évaluer des méthodes et solutions mises en place. Nous avons également comme client le laboratoire lui-même. Quand le laboratoire montre un besoin de programmation ou d'autres tâches d'ingénierie pour un des projets en cours, il fait appel à notre cellule si cela reste dans nos domaines de compétences. D'autres clients plus ponctuels peuvent aussi faire appel à notre cellule pour la mise en place d'outils liés à la vision par ordinateur et à l'image plus généralement.

Cet environnement facilite donc les échanges avec le laboratoire, ce qui fluidifie l'avancement des projets de recherche, mais apporte aussi à notre cellule un lien fort avec les thématiques de recherche actuelles. C'est pour nous un argument très important, car cela montre la possibilité de travailler sur des solutions innovantes. Ce lien est donc bénéfique pour les deux parties.

La cellule est actuellement composée de 8 membres, ce chiffre évolue fréquemment, notamment du fait de l'arrivée de stagiaires ou selon la durée des contrats en cours.

3. Organisation du projet

Ce chapitre cherche à répondre à une question : comment une petite cellule, à l'interface entre un laboratoire de recherche et une entreprise, s'organise-t-elle pour mener un projet complexe, avec les contraintes de ressources et de coordination que cela implique ? Les sections qui suivent y répondent en présentant le cadre du projet, les rôles de chacun, puis les méthodes et outils de travail au quotidien.

3.1. Contexte du projet

Ce projet est en lien avec Amazon Elemental, filiale d'Amazon. Ce groupe s'intéresse aux problématiques liées à la vidéo, notamment du fait des différents services qu'il propose, tel que Prime Video. Ce projet prend part à une collaboration à plus long terme et fait suite à d'autres projets en lien avec cet organisme.

Ce projet s'inscrit aussi dans la dynamique de la cellule. Pendant son déroulement, il occupe une place importante au sein de l'équipe, il mobilise plusieurs membres et s'appuie sur les compétences cœur de la cellule.

Ce projet a pour ambition d'étudier le sujet de l'IA dans le cadre de l'optimisation vidéo. Cela prend la forme d'une preuve de concept, où les études réalisées seront présentées à l'entreprise cliente afin de définir la faisabilité d'une telle optimisation. Pour ce faire, différents jalons sont posés : quels outils permettent au mieux d'évaluer la qualité d'un contenu vidéo ? Par la suite, des FOM (*Figure Of Merit*) devront définir les méthodes d'évaluation de la réussite d'une optimisation. Ces premiers jalons donnent une base solide qui sera réutilisable pour les étapes d'optimisation. D'autres aspects du projet consistent aussi en l'évaluation des méthodes d'optimisation par IA. C'est d'ailleurs dans cette partie que ce projet de fin d'études prend place : le but est de définir les outils possibles pour ce genre d'optimisation, et de répondre aux différentes problématiques posées par les outils de codage vidéo classiques pour l'apprentissage. Il sera alors question d'implémenter des solutions et d'évaluer leurs performances et leur pertinence dans le projet. Ces tests sont aussi liés à l'évaluation des différents outils de mesure de qualité, car ils seront la clé d'un apprentissage réussi. Par la suite, une fois les différentes études menées sur l'outillage nécessaire à une optimisation réussie, l'objectif final sera de mettre ces outils en œuvre sur des cas contrôlés de vidéo, afin d'évaluer les optimisations obtenues. Il est important de rappeler que ce PFE s'intègre dans le projet et que différentes contributions seront réalisées plus tard, dans la suite de celui-ci.

3.2. Planning

Le projet est structuré autour de trois Work Packages complémentaires, s'étalant d'avril 2026 à mars 2027.

L'articulation globale des tâches ainsi que l'enchaînement des différents jalons de validation sont détaillés dans le diagramme de Gantt :

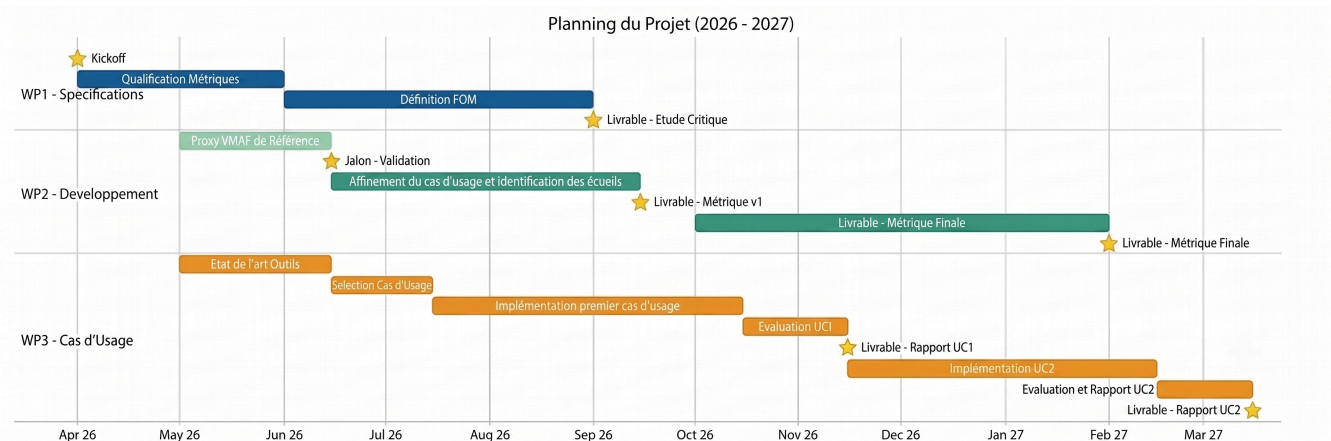


Fig. 12. – Planning global du projet (2026-2027)

3.3. Les membres du projet

Dans la section précédente, nous avons évoqué le nombre de membres de la cellule, ici, nous verrons combien de personnes participent à ce projet et les rôles de chacun.

Patrick LE CALLET (Responsable scientifique)

Il assure le bon déroulement du projet et les discussions avec le client afin d'aboutir aux exigences du projet de départ. Il est aussi un élément moteur des idées amenant à ce projet.

Pierre LEBRETON (Ingénieur et responsable de cellule)

Encadrant des membres de la cellule, et donc des participants au projet, il est au quotidien le garant de la qualité des actions liées au projet, il en est aussi le référent technique. Son rôle est polyvalent : il travaille sur des implémentations techniques mais aussi sur la gestion des différents aspects du projet.

Lina GUEMBRI (Ingénieure)

Son rôle se trouve majoritairement dans l'évaluation des **métriques** de qualité vidéo, l'étude de leurs défauts et les solutions permettant d'y échapper.

Jipeng XIA (Stagiaire)

Il travaille sur la thématique des outils d'optimisation de compression vidéo par filtrage IA, en particulier en étudiant l'état de l'art des solutions actuelles et leurs implémentations.

Mon rôle

En tant qu'alternant, ce projet a commencé pour moi par une première étude sur l'utilisation d'une **métrique** de qualité vidéo dans le cas de l'entraînement d'un **codec neuronal**. L'objectif était de faire ressortir de possibles améliorations mathématiques de cette métrique afin d'en améliorer la pertinence et son utilisation durant l'apprentissage. C'était un premier pas dans ce domaine, qui m'a permis d'apprendre de nombreuses notions importantes.

Par la suite, dans le cadre de mon PFE, je travaille majoritairement sur les analyses et implémentations d'outils d'optimisation, plus particulièrement sur l'aspect **proxy** de **codec** afin de contourner les limitations des outils classiques, en d'autres termes, créer un jumeau des outils classiques qui guidera l'apprentissage. Si l'on s'en fie au planning, cette tâche fait partie du Work Package n°3 concernant l'étude des cas d'utilisation des métriques, dans ce cas : le filtrage d'images en pré-compression. La thématique des outils de mesure de qualité vidéo étant directement liée à ce sujet, ma mission réside aussi dans l'étude des meilleurs outils pour ce cas d'usage et de leurs spécificités, ce qui fait le lien avec les différents autres jalons du projet.

3.4. Estimation horaire du projet

Ce projet a débuté pour moi courant mars, la date de rendu du document étant fixée mi-juillet, ce projet de fin d'études se déroule durant environ 4 mois. Ce qui représente environ 80 jours ou environ 560 heures de travail.

Étapes	Jours
Etat de l'art et apprentissage continu (compression, deep learning, méthodes, métriques)	15 j
Étude et analyse des métriques de qualité (corrélations, choix du guide, test d'optimisation)	5 j
Implémentation des solutions : Code, test (concluants ou non), debug	25 j
Expérimentations : entraînements/itérations, tests, analyse des résultats	20 j
Réunions POP, points d'équipe, séminaires, échanges encadrants	5 j
Rédaction du mémoire, figures, documentation projet, présentation réunion	10 j
Total	80 j

Tableau 1. – Répartition prévisionnelle de la charge de travail du PFE (en jours).

3.5. Outils et méthodes de travail

3.5.1. Méthodes de suivi et de travail

Au sein de la cellule, les projets ne regroupent que peu de personnes à la fois, et les missions peuvent parfois ne pas nécessiter une communication quotidienne poussée. Il reste cependant important de garder en tête les avancées de chacun, c'est pourquoi nous tenons des réunions hebdomadaires. Nous suivons la méthode POP, qui simplifie la gestion des réunions en les rendant plus dynamiques. Tous les lundis, face à un tableau prévu à cet effet, représentant la semaine actuelle et la semaine passée, nous précisons ce que nous avons réalisé la semaine précédente et ce que nous envisageons de faire pour la semaine en cours. On utilise aussi des post-it qui permettent de garder une trace des points clés évoqués et de suivre ce qui était prévu et ce qui a été fait d'une semaine à l'autre. C'est un moment où l'on peut plus facilement débloquer des situations, se donner des conseils et établir une organisation des tâches plus cohérente, car tous les membres sont alors disponibles.

Le reste du temps, les échanges concernant les différents projets sont plus informels, facilités par notre proximité dans les locaux. Il est aussi fréquent de faire des points ou des présentations pour mettre au clair un avancement, des idées ou des besoins spécifiques pour un projet, des moments essentiels quand le projet regroupe plusieurs personnes.

Les échanges se font majoritairement en français, mais ponctuellement en anglais, en fonction de l'aisance de chacun avec le français.

3.5.2. Outils de communication et de suivi

Notre cellule étant de petite taille, la communication y est facilitée, nous travaillons tous dans le même bureau. Cependant, nous faisons partie de deux autres organismes, le laboratoire de recherche, avec lequel nous partageons les mêmes locaux, et l'entreprise. Cela entraîne donc des besoins de communication et de gestion par des canaux différents. Concernant le laboratoire, nous avons des canaux liés à l'université, notamment l'outil Mattermost, qui permet de réaliser des échanges en ligne sous forme de chat, en groupe (notamment pour les différents projets) ou seul. D'autres outils liés à l'université nous sont mis à disposition : UnCloud, qui permet d'avoir un stockage cloud et de partager facilement des documents volumineux, Webmail, un service de mail en ligne en lien avec l'université, Glicid, un cluster de calcul partagé disponible pour la recherche, en particulier dans la région nantaise. Le lien avec l'entreprise est, lui, plus distant, des outils et protocoles sont mis en place pour suivre l'évolution des projets par les responsables et services de gestion de l'entreprise. Lucca permet la gestion des congés et autres absences ainsi que le partage de documents RH aux employés, Laboxy permet d'attribuer les heures effectuées aux projets concernés.

3.5.3. Outils internes

Au sein du laboratoire, et en particulier de la cellule, nous possédons aussi des serveurs de calcul qui permettent de répartir les ressources entre les membres de l'équipe et, en particulier, de stocker des dossiers volumineux, notamment des vidéos sources qui peuvent parfois devenir très encombrantes. Ces outils sont importants, mais demandent une organisation particulière pour une utilisation par plusieurs personnes : création de sessions différentes, limites d'utilisation des ressources processeur ou graphique par chacun. Le nombre de membres au sein de l'équipe ayant augmenté, ces outils internes ne suffisent plus pour que chacun puisse les utiliser quotidiennement, mais ce ne sont pas les seuls outils à disposition.

3.5.4. Adaptation et impact des outils utilisés

Parmi les outils utilisés durant ce projet, l'un en particulier permet de nous affranchir d'un besoin matériel important : Glicid. C'est un cluster de calcul partagé, accessible à un ensemble de projets de recherche et aux entreprises qui en paient l'accès. Glicid fournit différents environnements permettant d'accéder à des processeurs puissants ou à des cartes graphiques, deux moyens d'effectuer un grand nombre de calculs et de faire exécuter différents algorithmes. C'est essentiel pour travailler avec des outils d'intelligence artificielle comme nous le faisons. Cette plateforme permet donc de centraliser les ressources matérielles nécessaires à de nombreux projets. Cela rend ces ressources plus accessibles financièrement, mais limite aussi l'impact écologique de chaque projet. Chaque environnement est utilisé pleinement, ce qui permet d'en optimiser l'utilisation, quand un test est terminé, un nouveau peut être lancé. De plus, les outils proposés évoluent, ce qui mutualise les besoins et facilite l'accès à de nouvelles technologies coûteuses.

Cet outil nécessite cependant une adaptation pour une utilisation optimale : différents environnements à comprendre, et la manière de bien formuler la demande pour un environnement qui réponde aux besoins du test en cours. Cela fait donc partie de nos missions, s'adapter aux outils utilisés et à leurs évolutions au fil du temps.

Pour notre cellule, on peut facilement se partager des astuces ou bonnes pratiques pour mieux appréhender ce genre d'outil. C'est devenu un indispensable pour certains membres de l'équipe, et cela permet de mieux gérer nos ressources internes. Glicid est aussi maintenu, ce qui facilite la tâche, car pour des ressources internes, cela demande plus de travail en cas de problème.

Cependant, ces maintenances imposent aussi des moments d'arrêt de l'outil, un grand nombre de jours durant ce projet ont été privés de cette ressource pour maintenance ou inaccessibilité de la plateforme notamment lors des périodes de canicule. Cela limite l'organisation des tâches et peut parfois ralentir l'avancée de certains projets.

De manière générale, c'est un outil atypique qui modifie la manière de travailler et demande de l'adaptation pour certains mécanismes spécifiques et contraintes liées à l'accès à la plateforme, l'organisation au quotidien en est dépendante.

3.5.5. La vie du laboratoire

Partageant les mêmes locaux et faisant partie de l'équipe IPI, nous participons aussi aux différents moments conviviaux de l'équipe, notamment les repas et les séminaires. Il est fréquent que des séminaires soient organisés, où doctorants, chercheurs ou invités présentent leurs travaux. Ces présentations se déroulent en anglais pour que tout le monde puisse suivre, car le laboratoire accueille souvent des stagiaires étrangers.

Ces moments profitent à tous. Pour ceux qui présentent, c'est l'occasion de valider leurs propos, de recueillir des idées d'autres chercheurs et de gagner en assurance. Pour ceux qui écoutent, c'est une façon d'aborder des concepts complexes et de pratiquer l'anglais sur des sujets précis et techniques.

En apparence anodins, ces moments illustrent assez bien ce que la littérature en management appelle l'apprentissage par l'action (*execution-as-learning*), par opposition à une logique de pure exécution [8]. Edmondson montre notamment qu'une équipe apprend mieux quand elle dépasse ses frontières internes (distances physiques, différences de statut, écarts de connaissance), par exemple via des temps d'échange réguliers. Les séminaires jouent exactement ce rôle, ils font circuler les connaissances entre des profils et des projets différents. Une étude menée sur 90 équipes, citée par ces travaux, observe d'ailleurs que les questions et interruptions lors des réunions améliorent le transfert de connaissances et l'acquisition de nouvelles pratiques. Cela suppose toutefois un climat où questionner n'est pas perçu comme une marque d'ignorance, ce que ces auteurs nomment la sécurité psychologique. En faisant le parallèle avec nos réunions, on retrouve une ambiance ouverte pour ces présentations, car c'est l'objectif, questionner sur un sujet que l'on découvre souvent durant la présentation. Il est parfois difficile de rendre les questions utiles pour le sujet en lui-même, et pas uniquement de les orienter pour notre propre compréhension de la présentation. Ce qui fait que malgré ce climat ouvert il est parfois difficile pour tout le monde de trouver cette sécurité psychologique mais les échanges et questions profitent à tous et la variété de sujets apporte une vision globale de la recherche. Les thématiques abordées s'éloignent parfois de nos domaines d'expertise mais c'est finalement le point le plus intéressant, notamment pour une équipe jeune, de développer son ouverture d'esprit face aux problématiques du monde dans lequel nous vivons.

3.6. Limites et adaptation

La première limite est celle de la connaissance. Bien que des bases existaient en compression vidéo et en apprentissage automatique grâce aux projets précédents et cours suivis, les connaissances liées à ces deux domaines réunis représentaient un gap important à combler. Une partie du travail a donc

consisté à réaliser de la bibliographie, des tests et inévitablement des erreurs, qui font partie du processus d'apprentissage.

La première étape a donc été d'étudier les solutions existantes et de comprendre le domaine. Cette compréhension ne s'est pas arrêtée là, elle s'est aussi construite au fil du projet, au fur et à mesure des implémentations et des résultats.

Les indicateurs de réussite ont eux aussi évolué. Au départ, peu d'informations étaient disponibles pour réaliser ces choix. On s'est appuyé sur la littérature et les connaissances de l'équipe. L'avancement du projet a ensuite permis de valider ou d'ajuster ces outils d'évaluation, notamment grâce aux analyses de corrélation qui seront présentées dans le chapitre concernant l'implémentation.

L'implémentation a suivi une logique d'essais et d'erreurs, avec des tests parfois non concluants qui ont néanmoins permis de mieux comprendre les mécanismes du deep learning dans ce contexte.

La deuxième limite se trouve au niveau des ressources matérielles. Comme évoqué plus tôt, les ressources de calcul ne sont ni illimitées ni toujours accessibles. Les pannes ou périodes d'inaccessibilité de Glicid ont régulièrement imposé de réorienter le travail vers d'autres tâches : bibliographie, rédaction, analyse, tout ce qui ne nécessite pas de ressources de calcul importantes.

Enfin, la limite de temps est réelle, ce PFE s'inscrit dans un projet encore récent, et certaines questions n'ont pas encore de réponse définitive.

Face aux limites de temps et de disponibilité des outils, des choix ont été faits, privilégier des tests plus simples mais solides pour parvenir à des premiers résultats évaluables, plutôt que de s'éparpiller sur des pistes trop nombreuses.

4. Compression et qualité vidéo : défis et solutions

4.1. Contexte de l'équipe et formation

On peut ici se poser une question : comment une équipe jeune vit-elle et monte-t-elle en compétence sur des thématiques de pointe comme le deep learning et la qualité vidéo, entre veille permanente, partage de connaissances et accompagnement ?

La jeunesse de l'équipe est un point à prendre en compte. Cela peut être vu comme un atout pour se tourner vers l'innovation et faciliter l'acceptation aux changements, mais cela demande aussi une montée en compétence rapide sur des sujets d'expertise pointus pour répondre aux attentes du projet. La compression vidéo est un domaine riche en théorie et concepts complexes. Appartenir à une équipe experte facilite cette montée en compétence, mais une part du travail passe nécessairement par des recherches personnelles. Le deep learning associé à ce projet est tout aussi exigeant, et certaines compétences ne s'acquièrent qu'au fil des étapes du projet. Cette progression demande donc du temps, et l'évolution rapide de ces domaines impose une veille au quotidien.

La recherche sur l'adoption des technologies aide à nuancer l'idée, intuitive, qu'une équipe jeune serait simplement « moins réticente » face aux nouveaux outils. Dans leurs travaux, Morris et Venkatesh [9] observent surtout que les déterminants de l'usage varient avec l'âge, chez les travailleurs les plus jeunes, la décision d'utiliser l'outil est davantage portée par l'attitude, c'est-à-dire par l'utilité qu'ils lui perçoivent, tandis que pour les plus âgés pèsent davantage l'avis des pairs et la facilité de prise en main. Pour une équipe jeune comme la nôtre, cela suggère que l'adoption d'un nouvel outil ou d'une nouvelle méthode se joue surtout sur sa valeur perçue, donc concrètement ce que l'outil apporte. Ces résultats doivent toutefois être pris avec prudence, ils reposent sur un échantillon réduit, et les auteurs eux-mêmes rappellent qu'on ne peut distinguer nettement un effet propre à l'âge d'un simple effet de génération mais cela montre tout de même une tendance, liée à l'âge ou à la génération, ce qui peut confirmer qu'une équipe jeune, venant d'une génération proche les uns des autres, pourrait avoir un comportement commun face aux changements. Si l'on prend l'exemple concret au sein de notre équipe, nous avons vu arriver il y a quelque temps des accès directement fournis par l'entreprise pour un outil de chat par IA, qui s'intègre aussi directement dans des projets pour faciliter l'écriture de code informatique notamment. Avoir un outil commun permet de centraliser les connaissances sur ce domaine et les bonnes pratiques, mais surtout, de ne pas rendre le sujet tabou, ce qui pourrait amener à des comportements plus difficiles à contrôler. On entend actuellement parler de « shadow AI », phénomène émergent où l'utilisation de ce genre d'outils est réalisée par les employés sans validation de leurs supérieurs ou du service informatique. L'intérêt d'utiliser ces outils pour des jeunes ingénieurs est simple, cela facilite l'accès à un grand nombre de connaissances et simplifie l'implémentation pour tester des outils rapidement. L'acceptation semble

donc rapide car l'outil est vu comme utile. Cela renforce alors les résultats obtenus dans l'étude présentée. Il reste tout de même intéressant de rendre les pratiques plus contrôlées et raisonnées pour éviter des dérives, ce que l'entreprise essaie de faire au niveau global pour éviter que chaque cellule ait à le faire.

Au-delà de la perception des outils, la montée en compétence dépend fortement de l'environnement de travail. Les travaux d'Edmondson sur le « teaming » montrent qu'un environnement psychologiquement sécurisant, où chacun peut poser des questions, reconnaître ses limites et solliciter de l'aide sans crainte d'être jugé, accélère l'apprentissage collectif [8]. C'est un point clé pour nous, se sentir dans une équipe où d'autres ont probablement les mêmes questionnements, de par un manque d'expérience, pousse à partager les réponses ou les sources explicatives trouvées. Ce partage, associé à un investissement personnel et à une veille régulière, est ce qui permet de combler progressivement le manque d'expérience sur ces thématiques. Cela reste un processus qui prend du temps, notamment du fait de la complexité du domaine de la compression vidéo et, pour ce projet en particulier, de la complexité du domaine du deep learning.

4.2. Encodage : la réduction d'informations transmises

Le codage vidéo repose sur un grand nombre de principes parfois complexes pour réduire l'information transmise, le but ici sera seulement de donner les logiques de base et certains éléments importants qui permettront une meilleure compréhension du sujet et faciliteront l'identification des éléments d'une possible amélioration. Une vidéo est une suite d'images qui défilent, souvent très rapidement (plusieurs par seconde). On peut parfois avoir des vidéos qui contiennent 30, 60, voire 120 images par seconde (**FPS**), et plus dans certains cas, pour des types de contenus qui demandent une grande fluidité. Durant un court laps de temps, la scène ne change que très peu. C'est le point principal utilisé pour la compression vidéo, exploiter cette redondance d'informations de manière efficace.

Pour bien comprendre les différents concepts, il faut aussi avoir en tête qu'une image est décomposée en petits blocs, qui peuvent être de taille variable pour une même image. Dans les exemples, nous garderons une taille fixe pour faciliter la compréhension.

Pour illustrer un premier mécanisme utilisé pour exploiter le mouvement dans les vidéos, voici un exemple simple qui permet de comprendre la logique utilisée :

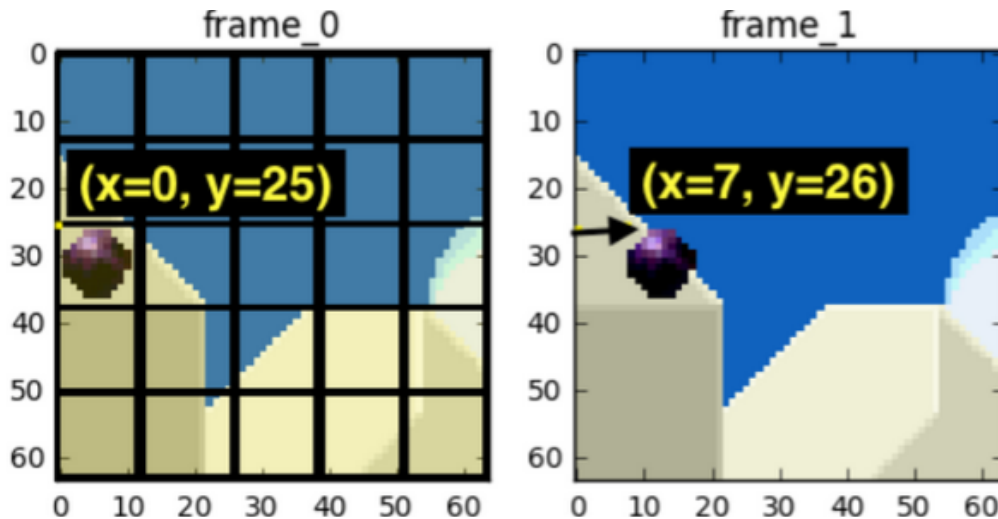


Fig. 13. – Exemple de mouvement prédit [5]

Cet exemple met en avant l'utilisation des différentes images d'une vidéo, on utilise alors les images précédentes pour prédire la position des éléments de l'image actuelle. Ici l'image change très peu, la boule est alors déplacée, et le reste de l'image sera réutilisé. Le premier point est la principale optimisation, elle utilise cette répétition d'informations, mais il faut tout d'abord transmettre une image clé, qui servira de point d'ancrage pour réaliser les prédictions suivantes. Une image clé ou « intra » est transmise pour chaque groupe d'images (**GOP**). Ces images clés doivent être transmises entièrement, ce qui peut parfois avoir un coût important. On utilise alors une autre forme de redondance, au sein d'une même image, des zones simples comme un ciel bleu pourront être transmises simplement, car les blocs voisins se ressemblent. Pour ce faire, il faut dérouler à partir des informations connues de l'image, qui sont déjà prédites. Le bloc en haut à gauche est transmis en premier, il servira alors de base pour la suite des prédictions qui vont dérouler jusqu'à terminer l'image.

Toutes ces prédictions sont réalisées avec comme contrainte de s'éloigner le moins possible de l'image d'origine, ce qui permet de choisir parmi les différentes options qui permettent de dérouler l'image.

Voici un exemple qui montre la manière dont cela est utilisé.

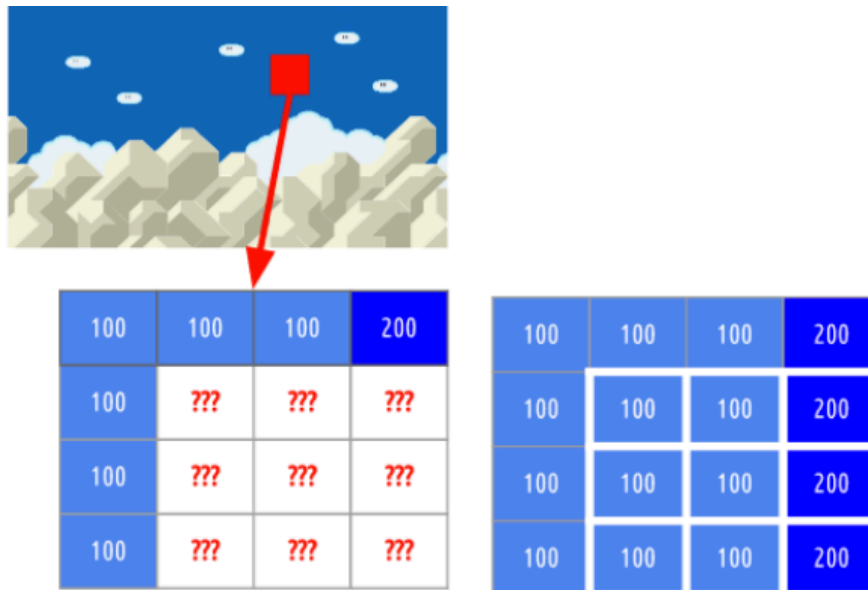


Fig. 14. – Exemple de la réutilisation de la partie de l'image connue [5]

Différentes options permettent de dérouler l'image, dans les outils récents il en existe de nombreuses mais pour comprendre la logique voici quelques exemples simples :

- La prédiction horizontale : On lisse de gauche à droite. L'encodeur considère que les pixels d'une ligne sont la continuité directe des pixels situés juste à gauche. Il recopie simplement les valeurs de la colonne précédente pour remplir le bloc.
- La prédiction verticale : On lisse de haut en bas. À l'inverse, l'encodeur estime que les pixels d'une colonne sont identiques à ceux de la ligne située juste au-dessus. Il « tire » l'information vers le bas pour prédire le contenu du bloc.
- La prédiction par moyenne globale : Dans les zones où les couleurs sont très homogènes (un aplat de couleur par exemple), l'encodeur calcule la moyenne des pixels voisins (en haut et à gauche) et applique cette valeur unique à tout le bloc.

Parmi ces trois options l'exemple imagé utilisait la prédiction verticale.

Une fois ces prédictions réalisées, l'image prédite n'étant pas parfaite, il manque des informations importantes à transmettre pour la corriger au mieux. Cette correction est appelée résidu : c'est la différence entre l'image prédite et l'image d'origine, autrement dit, c'est le reste des informations à transmettre pour arriver à l'image d'origine.

100	100	100	200
100	100	100	200
100	100	100	200
100	100	100	200

 $-$

100	100	100	200
100	100	100	200
100	100	100	200
100	100	120	210

 $=$

100	100	100	200
100	0	0	0
100	0	0	0
100	0	20	10

Fig. 15. – Exemple de calcul du résidu pour le ciel bleu [5]

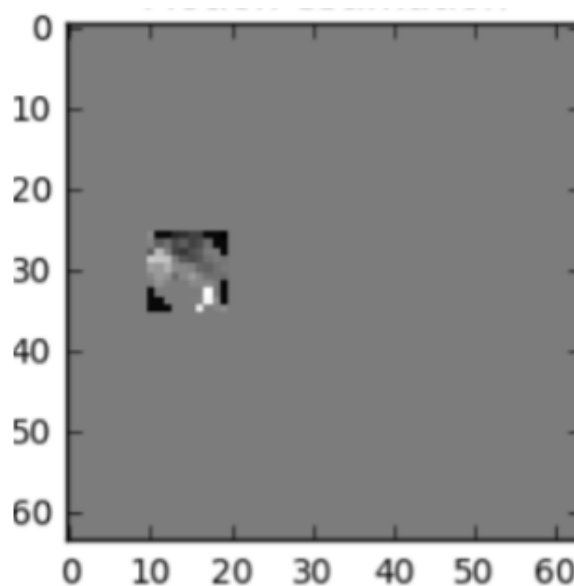


Fig. 16. – Exemple visuel de résidu pour l'exemple du mouvement de la balle [5]

Nous verrons dans la section suivante que ce résidu n'est pas transmis tel quel car il peut parfois contenir encore un grand nombre d'informations.

4.3. Utilisateur et compression ciblée

Au-delà de la redondance entre images et au sein d'une image, la compression exploite une autre source d'économie : les limites de la perception humaine. L'idée est simple, si l'œil ne perçoit pas une information, il est inutile de la transmettre fidèlement.

Deux mécanismes illustrent bien cette logique. Le premier repose sur la **DCT** (transformée en cosinus discrète), qui réorganise l'information d'un bloc par fréquences. Les zones lisses se concentrent dans les basses fréquences, les détails fins dans les hautes fréquences. Pour comprendre mieux la logique plus l'information (les pixels) change rapidement dans une zone plus la fréquence sera haute. Or, l'œil est bien moins sensible à ces hautes fréquences, on peut donc les représenter

plus grossièrement lors de la **quantification**, sans dégradation visible, ce qui allège fortement le poids du fichier. Cette importance peut être simplement évoquée par des exemples visuels, plus on monte dans les fréquences, moins l'information semble essentielle.

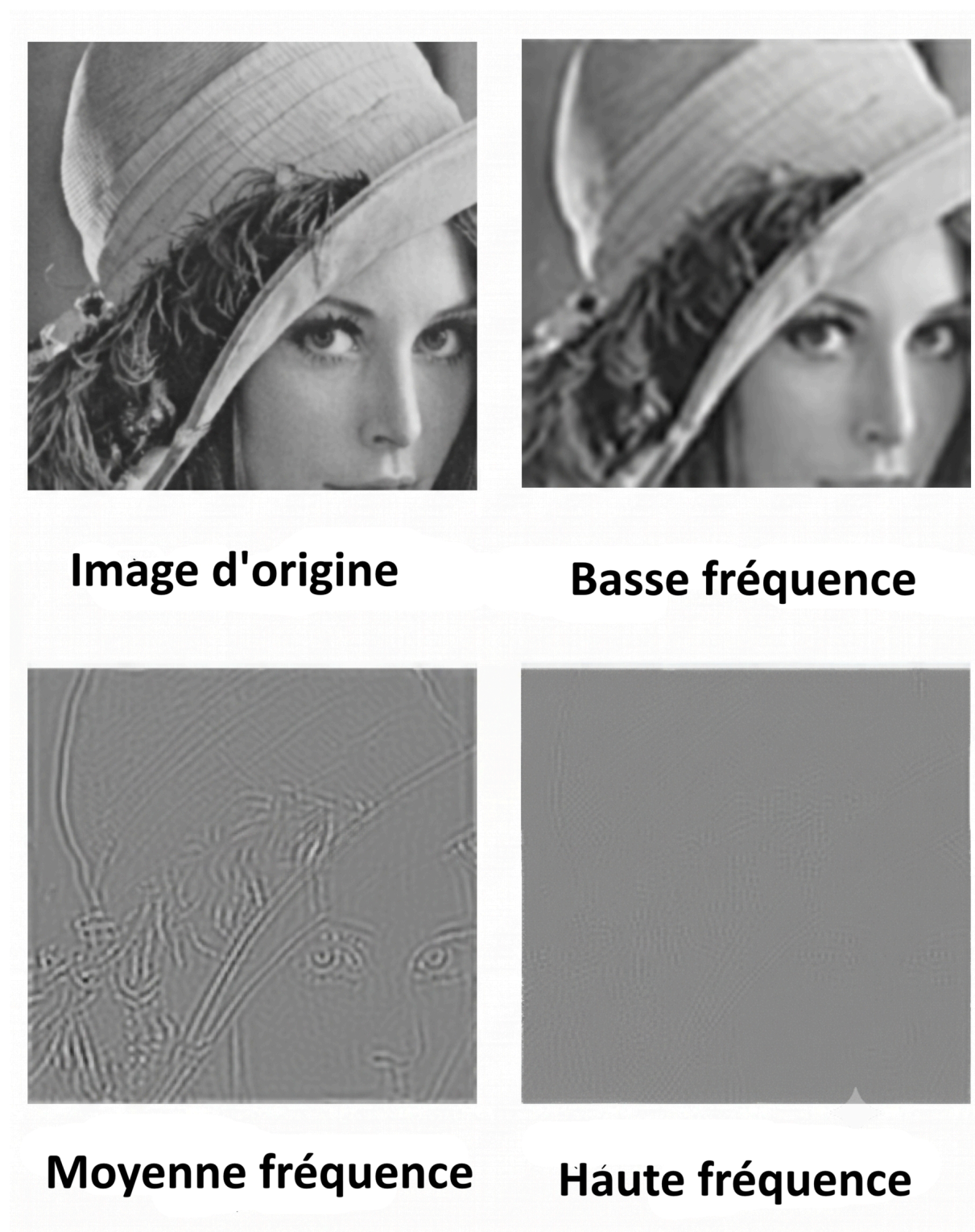


Fig. 17. – Illustration de l'importance de chaque domaine de fréquence pour la fidélité de l'image source [10]

Pour illustrer la simplification des données, voici aussi un exemple de l'utilisation de cette transformation **DCT**. Un résultat obtenu qui permet de trier les basses fréquences dans des valeurs qui seront mieux conservées (en haut à gauche) et les valeurs de hautes fréquences comme des détails (en bas à droite). Cet exemple illustre alors la **quantification** (arrondi), qui vient simplifier ces valeurs. Durant cette étape, on divise puis garde l'arrondi le plus proche, c'est cet arrondi qui fait perdre les détails.

120	40	1	0
45	3	0	0
-5	0	0	1
0	0	-2	0

 $\div \text{Qstep (10)} =$

12	4	0	0
4	0	0	0
0	0	0	0
0	0	0	0

Fig. 18. – Exemple de quantification venant simplifier les données obtenues après la transformation DCT (division + arrondi) [5]

Il est important de noter que la quantification n'a d'effet sur les données que si son paramètre « qstep » est supérieur à un. Dans le cas où il vaut un, les données restent intactes, on parle alors de compression sans perte. Cette transformation permet donc de cibler les fréquences de l'image que l'on veut simplifier, c'est là que repose la majorité des simplifications réalisées par les codecs pour obtenir des données plus compressibles, avec notamment davantage de valeurs simples comme des zéros. Un élément expliqué plus tard, en Chapitre 4.4.

Pour avoir une idée du résultat obtenu après quantification voici un exemple pour une zone d'image qui permet de voir précisément les détails.



Fig. 19. – Exemple de résultat après quantification, ici 67.1875% des coefficients sont supprimés [5]

On observe donc que cette étape cause une perte en qualité ou au moins en fidélité à l'image originale. On remarque cependant que malgré une perte de plus de la moitié des coefficients, l'image reste globalement similaire.

Le second mécanisme concerne la couleur. Une image peut être décrite en **RGB** (*Red, Green, Blue*), mais on lui préfère l'espace **YUV**, qui sépare la **luminance** (l'intensité lumineuse) des **chrominances** (la couleur). Là où, en **RGB**, cette luminance se retrouve dans chacune des composantes, ce qui empêche de la séparer du reste des informations.

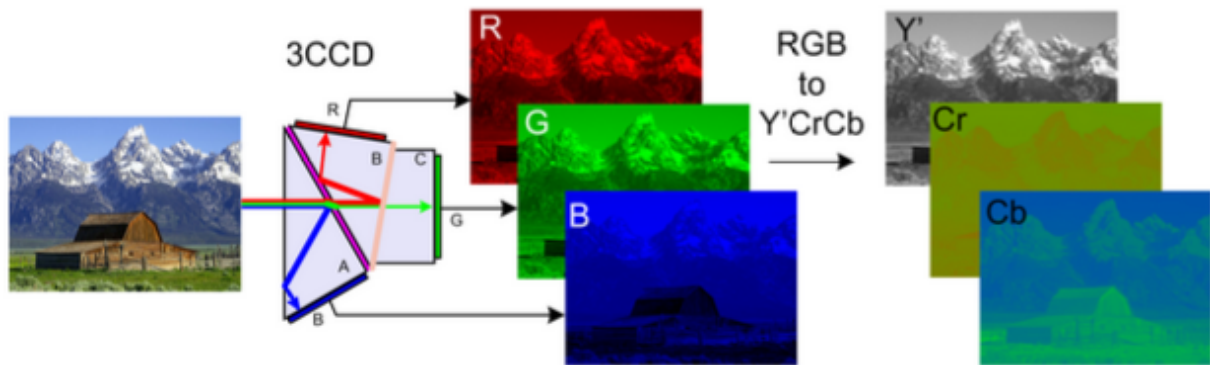


Fig. 20. – Exemple d'une image décomposée en **RGB** et en **YUV** [11]

L'œil étant beaucoup plus sensible à la luminance qu'à la couleur, on peut alors sous-échantillonner les informations liées à la couleur. On retrouve notamment le format 4:2:0, souvent utilisé pour les vidéos, où la couleur est transmise à une résolution réduite par rapport à la luminance. La perte est réelle, mais quasiment imperceptible.

Pour comprendre ce point et l'impact qu'il a sur les données, voici un exemple en image :

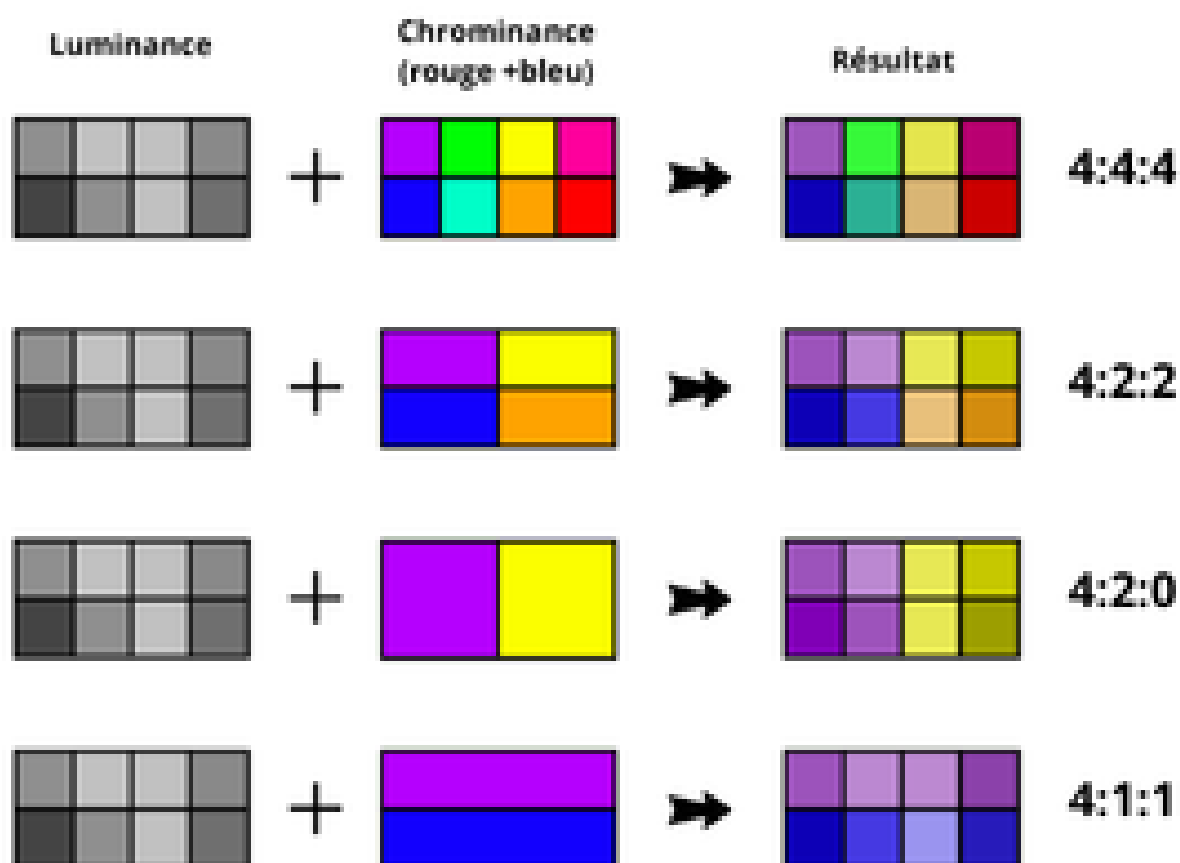


Fig. 21. – Exemple de réduction des données en 4:2:0 et autres formats, [12]

Pour garder l'exemple du format 4:2:0, on a donc deux couleurs qui sont en fait la concaténation des chrominances rouge et bleu ce qui en réalité, représente 4 informations.

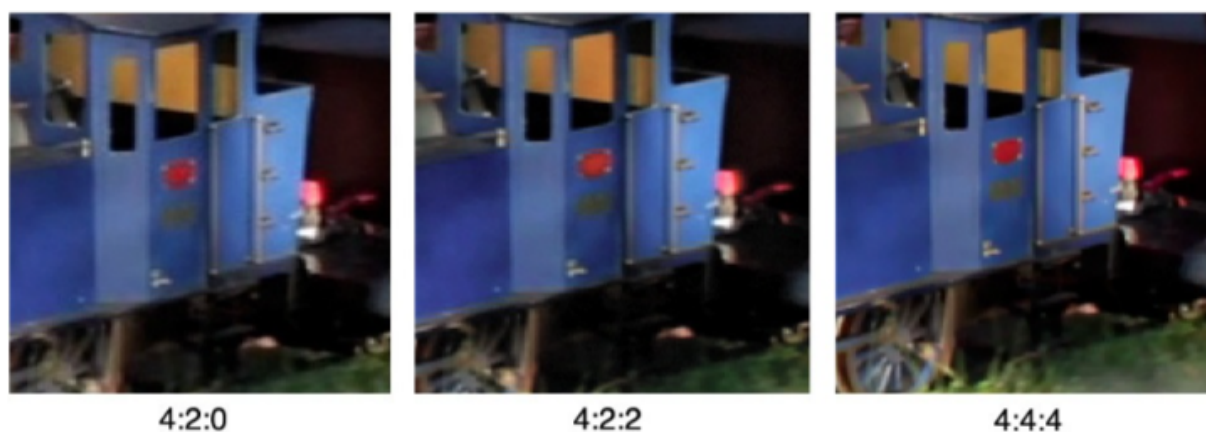


Fig. 22. – Exemple visuel en 4:2:0 et autres formats, [12]

On voit alors que malgré une réduction importante des données pour le format 4:2:0, qui supprime 75 % des informations de couleurs, on retrouve une image qui reste assez similaire à celle contenant toutes les informations (le format 4:4:4). Un autre exemple met en avant cette efficacité, avec cette

méthode (4:2:0) on garde 6 unités sur 12 (4 de luminance + 1 + 1 de chrominance), soit 50 %, on a donc réduit par deux les données sans perdre trop en qualité perçue.

Ces deux exemples montrent une chose importante pour la suite : la compression ne cherche pas la fidélité parfaite, mais la fidélité perçue. C'est exactement le terrain sur lequel se place ce projet, modifier l'image pour qu'elle coûte moins cher à coder, sans que l'utilisateur final ne le remarque.

4.4. Quantité d'informations des vidéos et transfert : logique et théorie

La théorie derrière le codage arithmétique est complexe, l'objectif de cette section est uniquement de mettre en avant la logique suivie, afin de comprendre comment une optimisation semble possible pour contrer les effets d'une compression parfois agressive.

Nous venons de voir brièvement comment les informations étaient transmises, cependant, ces informations peuvent faire varier grandement le poids des fichiers vidéo. Une logique simple repose sur la théorie de l'information : plus une information est prédictible, moins elle est lourde à transmettre. On comprend alors l'intérêt de la **quantification**, qui simplifie les valeurs, on retrouve par exemple un grand nombre de zéros, quasiment gratuits à transmettre.

C'est là qu'intervient le codage entropique, l'outil qui transforme réellement ces valeurs en représentation binaire (en bits). Dans **H.265**, ce rôle est tenu par le **CABAC**. Sans entrer dans le détail, il faut surtout retenir deux idées. D'abord, il est contextuel, pour coder une valeur, il regarde ses voisines déjà codées et s'en sert pour estimer ce qui va probablement suivre. Ensuite, il est adaptatif, au fil du codage, il met à jour ses probabilités en fonction de ce qu'il a réellement rencontré. Autrement dit, plus le contenu est régulier et conforme à ses attentes, plus il devient efficace.

On voit alors pourquoi avoir des valeurs simples est si important. Une suite de valeurs proches, répétées ou nulles est très prédictible, le **CABAC** lui attribue une forte probabilité et la code sur très peu de bits. À l'inverse, des valeurs dispersées et imprévisibles coûtent cher. C'est précisément ce levier que l'on cherche à exploiter, si l'on parvient, en amont, à rendre l'image plus simple à représenter une fois transformée et quantifiée (davantage de zéros, des coefficients plus réguliers), alors le codage final devient moins coûteux, à qualité visuelle comparable. C'est tout l'enjeu de l'optimisation visée dans ce projet.

4.5. Comprendre le déroulement entier

Les sections précédentes ont détaillé chaque mécanisme isolément. La figure Fig. 23 les remet dans l'ordre : de la vidéo brute au flux compressé (encodeur, colonne de gauche), puis du flux reçu à l'image reconstituée (décodeur, colonne de droite), afin de visualiser l'enchaînement complet et simplifié d'un **codec** tel que **H.265**.

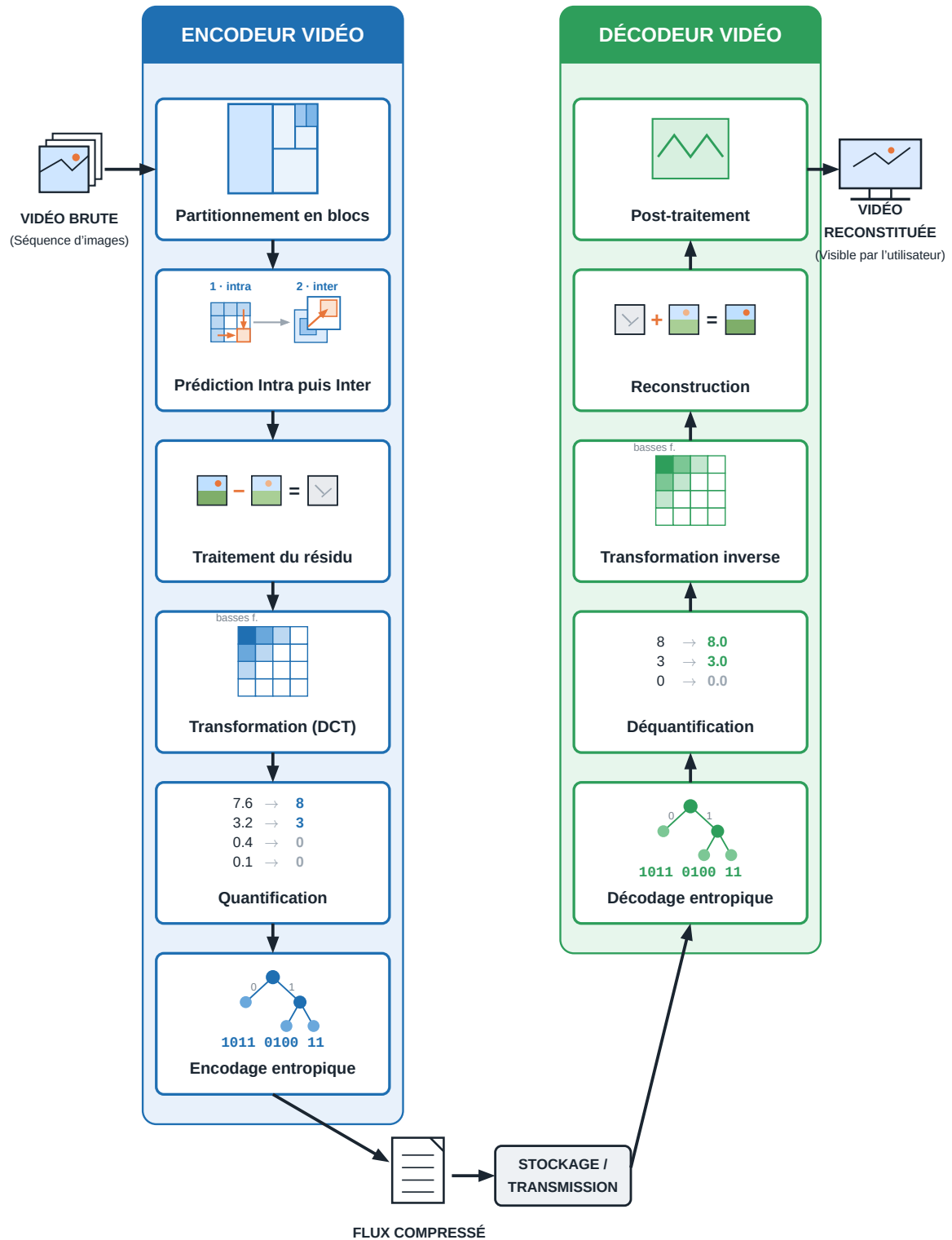


Fig. 23. – Déroulement complet d'un **codec**, de la vidéo brute au flux compressé (encodeur, colonne de gauche) puis du flux reçu à l'image reconstituée (décodeur, colonne de droite).

4.6. Évaluer le contenu vidéo

Si l'on veut optimiser une vidéo, encore faut-il pouvoir mesurer sa qualité. C'est un point central du projet, pour deux aspects, la mesure choisie servira de guide à l'apprentissage du filtre, elle se doit donc d'être un critère qui reflète la vision humaine, et durant l'évaluation, cela conditionnera aussi la pertinence des résultats obtenus.

La référence reste le jugement humain. En réunissant un panel d'observateurs et en moyennant leurs notes, on obtient un **MOS** (*Mean Opinion Score*), considéré comme la « vérité terrain » de la qualité perçue. C'est d'ailleurs l'une des expertises reconnues de notre équipe, sollicitée pour ce type de tests par de grands acteurs du secteur. Mais ces tests sont coûteux et lents : impossible de les utiliser pour guider, image par image, l'entraînement d'un réseau de neurones qui nécessite un grand nombre d'exemples.

On s'appuie donc sur des **métriques** objectives, c'est-à-dire calculées automatiquement par opposition à une évaluation subjective, réalisée par un utilisateur.

La plus ancienne, le **PSNR**, mesure simplement l'écart pixel à pixel avec la source, elle est facile à calculer, mais corrèle mal avec la perception humaine. Elle reste pourtant un indicateur intéressant pour mesurer les performances d'un codec, à quel point il reproduit exactement l'image d'origine.

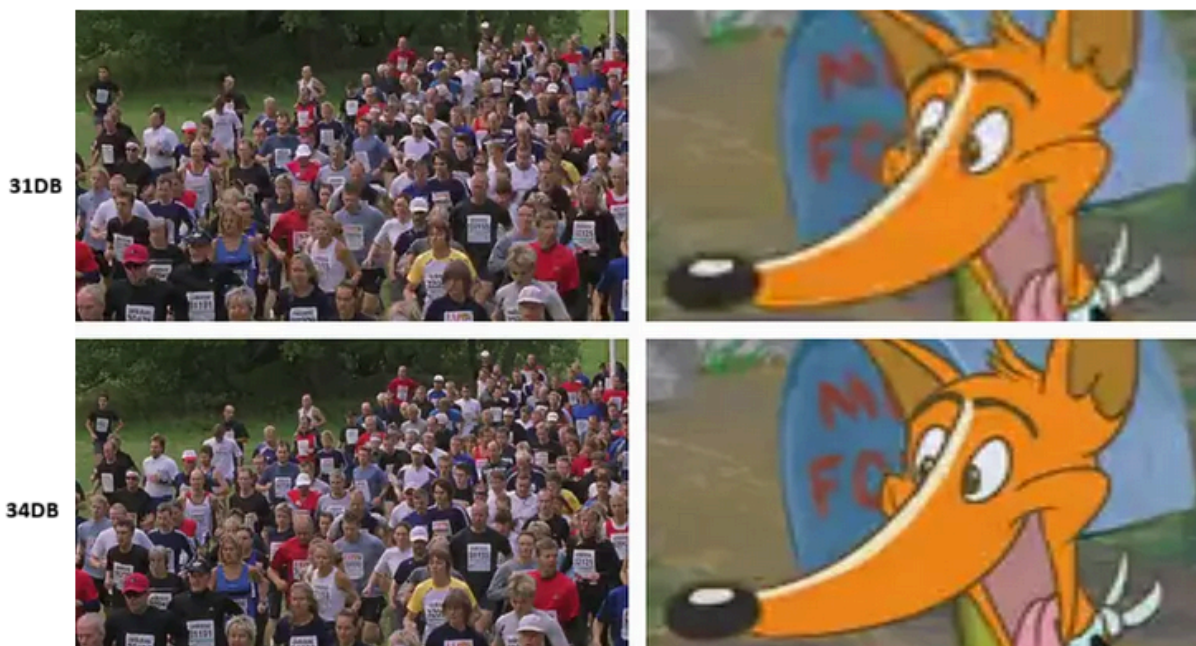


Fig. 24. – Exemple de la différence entre fidélité structurale et qualité perçue (score **PSNR** de 31 dB, mauvais, pour les deux images du haut, et de 34 dB, meilleur, pour celles du bas)

On voit, dans cet exemple, que la mesure de fidélité de l'image au niveau des pixels ne réagit pas du tout de la même manière selon l'image : un gain de 3 dB peut être invisible ou, au contraire, grandement améliorer la qualité.

D'autres métriques existent, comme **SSIM**, qui cherche aussi à comprendre les différentes structures de l'image, ou **VMAF**, qui cherche à se rapprocher du jugement humain en combinant plusieurs indicateurs. Ces métriques sont au cœur du projet, une métrique mal choisie peut conduire à optimiser dans une mauvaise direction. Elles ont aussi leurs défauts, à l'instar des codecs, elles sont rarement conçues pour des cas d'usage liés à l'apprentissage d'un réseau de neurones. Elles présentent des limites mathématiques et structurelles, et ne sont souvent prévues que pour juger des images classiques. Cela en fait un terrain propice à l'exploitation par une IA, qui peut apprendre à en exploiter les défauts pour améliorer le score sans réellement améliorer la qualité perçue.

Enfin, il faut garder en tête que ces métriques restent des approximations et ont chacune leurs limites. La validation finale d'une optimisation, une fois les outils suffisamment matures dans la poursuite du projet, passera par de vrais utilisateurs, c'est ce qui garantit la validité du projet, et c'est précisément le type de tests que notre cellule peut mener.

4.7. Les limitations pour l'apprentissage

Nous l'avons mentionné, les codecs classiques s'intègrent mal dans une chaîne d'apprentissage automatique. Avant de voir comment contourner ce problème, il faut comprendre d'où viennent ces limitations, en reprenant les étapes clés qui, dans leur version d'origine, ne peuvent pas faire partie d'un environnement d'apprentissage d'un réseau de neurones.

4.7.1. Limites mathématiques des outils pour l'apprentissage

Ces limites sont avant tout mathématiques. Pour qu'un réseau de neurones, dans notre cas, un filtre, apprenne, chaque opération de la boucle doit indiquer dans quelle direction ajuster les paramètres, c'est le rôle du « gradient », qui donne en quelque sorte le sens de la pente. Une opération est utilisable pour l'apprentissage si elle est différentiable, c'est-à-dire une fonction suffisamment régulière, sans cassure et dont on peut mesurer la pente.

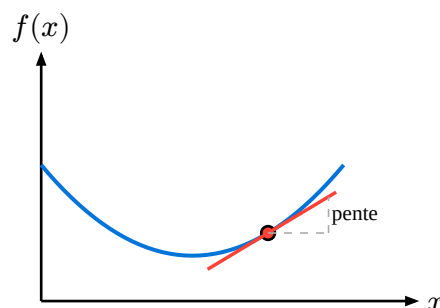


Fig. 25. – Cas d'une fonction différentiable : en tout point, on peut calculer la pente (le gradient), qui indique dans quel sens la fonction varie, et donc dans quelle direction ajuster le paramètre.

La raison principale de l'impossibilité d'utiliser ces outils de codage tels quels est qu'ils ne sont pas prévus pour cela, pas adaptés aux langages informatiques utilisés pour l'apprentissage. Ils sont aussi très optimisés pour des calculs rapides sur processeurs, mais bien moins dans des conditions d'entraînement. Ils possèdent de nombreux choix binaires qui constituent alors des murs à l'apprentissage, où l'on garde ou ne garde pas une information sans transition, ce qui provoque alors une cassure dans les fonctions et donc une impossibilité d'utiliser ces étapes dans une boucle complète d'apprentissage des poids d'un réseau.

On retrouve alors beaucoup de fonctions qui n'ont pas les propriétés adéquates. Certaines présentent des zones plates, où une petite variation de l'entrée ne change rien à la sortie, la pente y est nulle et n'indique aucune direction. D'autres présentent des cassures franches, où la pente n'est tout simplement pas définie.

Les **codecs** vidéo regorgent de ce type d'opérations. La **quantification**, par exemple, repose sur un arrondi, sa courbe est un escalier, plat entre deux marches, donc de gradient nul presque partout. La sélection du meilleur bloc lors de la prédiction repose, elle, sur ce qu'on appelle un « argmin », un choix qui ne garde qu'une unique solution parmi des candidats distincts et qui ne fournit donc aucune pente exploitable.

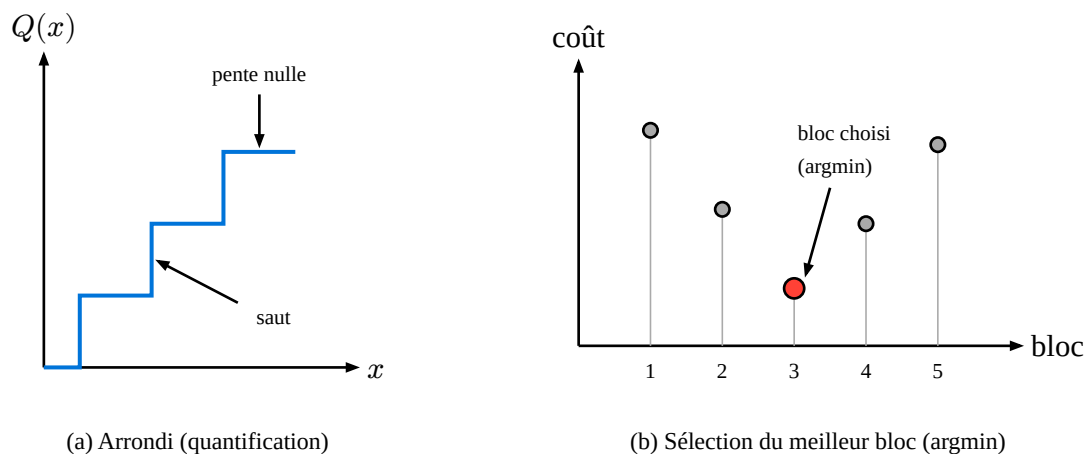


Fig. 26. – Deux opérations non différentiables typiques d'un **codec**. À gauche, l'arrondi de la **quantification** : sa courbe en escalier est plate entre les marches et discontinue aux sauts. À droite, la sélection du meilleur bloc par argmin : le choix n'est pas continu, il saute d'un bloc à l'autre sans fournir de pente exploitable.

C'est notamment pour ces raisons qu'il devient complexe de reproduire fidèlement un apprentissage complet qui guiderait vers les meilleures options pour ces fonctions. Nous verrons par la suite qu'il est possible de supprimer certaines opérations et de les rendre invisibles, cependant, l'optimisation devient alors aveugle à des éléments qui sont au cœur des choix lors de la compression. Il faut alors trouver le juste milieu pour que l'environnement d'apprentissage regroupe et puisse comprendre les éléments essentiels de cette optimisation.

Le défi majeur se trouve donc dans la création d'une version repoussant ces limites, mais qui s'approche de la réalité, ce qui est très complexe notamment de par les optimisations poussées des codecs, des optimisations compliquées à reproduire d'autant plus qu'elles sont souvent non optimisables.

4.8. Solutions existantes

La mise en place d'un terrain d'optimisation fiable : un **proxy** (jumeau du codec cible) est un des piliers du projet, c'est lui qui permettra d'entraîner correctement le filtre et, plus tard, possiblement d'autres outils destinés à assister la compression. Tester différentes méthodologies de proxy était donc un premier point d'étape essentiel, cela s'intègre naturellement dans le cadre d'un projet de fin d'études. La partie scientifique se penchera donc en particulier sur ce point en apportant aussi d'autres aspects du projet mais à ce stade précoce, il est d'abord essentiel de comprendre les différentes possibilités avant de définir la méthode qui sera utilisée pour l'implémentation finale.

Dans cette section, nous verrons différentes solutions existantes qui serviront de références techniques pour les tests mis en place.

4.8.1. Deep learning et méthodes de remplacement

Lors de l'apprentissage d'un réseau de neurones, dans notre cas le **filtre**, le code qui réalise cet apprentissage est rempli d'opérations mathématiques qui sont les piliers de l'optimisation des paramètres du réseau. Pour permettre à chaque opération d'avoir sa place dans cette optimisation, un graphe de calcul est créé, il regroupe les différents calculs exécutés durant la boucle.

Ce graphe se parcourt dans les deux sens. À l'aller (le **forward**), on calcule normalement le résultat à partir des entrées. Au retour (le **backward**), on remonte le graphe en sens inverse pour calculer, pour chaque opération, le gradient, c'est-à-dire la contribution de chacune des opérations à l'erreur finale qui remonte ce graphe. C'est cette seconde étape qui permet d'ajuster le réseau et c'est ici qu'il est nécessaire que les opérations mathématiques soient dérivables sinon l'erreur ne pourra pas être corrigée (le gradient sera bloqué).

Séparer ces deux étapes ouvre une possibilité intéressante : on peut intervenir sur le **backward** indépendamment du **forward**. Deux outils en découlent, le premier consiste à « détacher » une partie du graphe, de sorte que l'opération ne soit pas prise en compte : la valeur est bien calculée à l'aller, mais on ne remonte pas par ce chemin. La Fig. 27 illustre ce principe sur un calcul simple.

Le second, plus puissant, consiste à remplacer le comportement d'une fonction au **backward**. On exécute l'opération réelle à l'aller, mais on la remplace par une approximation optimisable au retour. C'est le principe du **STE** (*Straight Through Estimator*), pour un arrondi, par exemple, on applique

le vrai arrondi au **forward**, mais on fait « comme si » la fonction était différente au **backward**, on utilise alors une fonction dérivable, ce qui laisse passer un gradient exploitable.

C'est exactement ce mécanisme qui rend possible l'usage d'opérations bloquantes, voire d'un codec entier, au cœur de l'apprentissage, on garde le comportement réel là où il compte, tout en fournissant au réseau une pente artificielle mais utile pour apprendre.

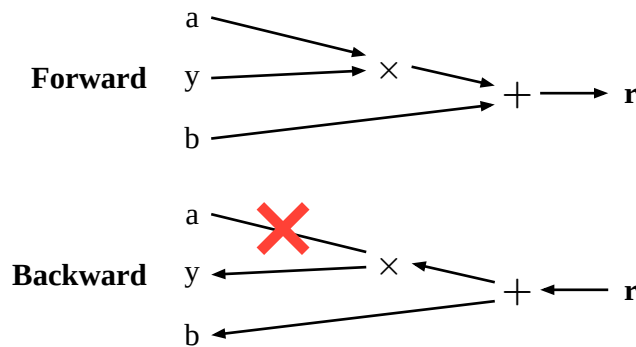


Fig. 27. – Graphe de calcul : $r = ay + b$ (a est détaché, son gradient est bloqué au *backward*)

4.8.2. Les méthodes d'optimisation existantes

Le filtrage de prétraitement par IA est aujourd'hui une thématique bien présente dans la littérature. Pour autant, il reste difficile d'établir quelle méthode est la plus performante pour remplacer les outils de codage classiques durant l'apprentissage, et comment guider au mieux ce dernier. On peut regrouper les travaux existants autour de deux grandes familles de **proxy** : ceux qui reconstruisent une version simplifiée et différentiable d'un codec classique, et ceux qui entraînent un **codec neuronal** à imiter le comportement d'un codec cible. Nous présentons ici trois travaux représentatifs de ces deux approches.

Neural Wrapping (CVPR 2025) — un codec neuronal comme proxy [13]

Ce travail, mené par Sony, s'intéresse à du contenu de type jeu vidéo et à du streaming à des débits représentatifs de la diffusion qui ressemble à notre cas d'utilisation. L'architecture combine un filtre de prétraitement et un filtre de post-traitement (un « neural wrapper ») encadrant le **codec**. Ils utilisent des filtres légers, l'encodage lui-même utilisant au contraire des presets lents et de haute qualité, ce qui se rapproche des conditions que l'on souhaite tester pour ce projet.

Pour contourner la non-différentiabilité des codecs classiques, les auteurs n'essaient pas de reproduire les étapes du codec, ils entraînent un **codec neuronal** à l'imiter. Cet alignement se fait en deux phases. On aligne d'abord les images reconstruites par le proxy sur celles produites par le vrai codec, puis on apprend la distribution du débit avant de la recaler sur le débit réel mesuré. Le modèle apprend donc à reproduire des images puis à comprendre comment le codec de base estime le coût de ces images. Pendant l'apprentissage de bout en bout, le proxy est réajusté à chaque itération, et

une astuce de *stop-gradient*, proche dans l'esprit du **STE** présenté plus tôt, il permet de transmettre au post-traitement les vraies images du codec tout en faisant circuler le gradient à travers le proxy neuronal.

L'intérêt principal est de repartir d'outils de codage neuronaux existants, réentraînaibles sur ses propres données, et d'obtenir une architecture relativement simple à appréhender. Malgré une différence de fonctionnement avec un codec classique, les résultats montrent que l'approche reste cohérente une fois le filtre appliqué sur le vrai codec. Les auteurs rapportent un gain moyen de l'ordre de $-18,5\%$ de **BD-rate** (le **BD-rate** mesure l'économie moyenne de débit à qualité égale, plus il est négatif, mieux c'est), et vérifient surtout que débit et distorsion du proxy et du codec cible se corrélaient fortement après l'alignement, condition nécessaire à un apprentissage par gradient.

Limites. La première limite tient au paradigme lui-même : les fonctions du codec (transformée, **quantification**, codage entropique) sont ici entièrement remplacées par des réseaux de neurones, qui réagissent différemment d'un vrai codec. On peut donc penser qu'un proxy plus proche du fonctionnement réel constituerait un meilleur simulateur pour l'apprentissage, c'est précisément l'une des pistes explorées dans ce projet. Ensuite, réutiliser un codec neuronal existant contraint aux points de fonctionnement pour lesquels il a été conçu car il est entraîné à une qualité fixe, des variantes à débit variable apparaissent mais sont encore peu accessibles pour un réentraînement. De plus, l'alignement sur un nouveau codec cible reste coûteux, les auteurs notent eux-mêmes un surcoût de calcul important pour l'alignement continu. Enfin, et c'est central pour notre projet, une grande partie des gains repose sur le post-traitement, leur étude d'ablation montre que le prétraitement seul dégrade même certaines **métriques** de fidélité (**SSIM**, **MS-SSIM**), et que seul l'entraînement conjoint du pré- et du post-traitement donne des gains cohérents sur l'ensemble des **métriques**. Or, un post-traitement côté client est difficile à déployer dans notre contexte (cf. les contraintes matérielles évoquées plus haut Chapitre 1.1.1). Un dernier point essentiel est leur choix de métriques durant l'apprentissage, ils utilisent une des composantes de **VMAF**, ce qui facilite probablement les gains face à cette mesure.

DPP (CVPR 2021) — un codec simplifié et différentiable [14]

On retrouve ici du filtrage neuronal, mais uniquement en prétraitement : aucune composante n'est ajoutée côté décodeur, ce qui rend la méthode directement déployable. Pour remplacer le codec cible, les auteurs « virtualisent » les briques essentielles d'un codec hybride en versions différentiables : prédiction inter/intra, sélection du meilleur bloc, transformée fréquentielle (la **DCT**) et **quantification**. Les opérations bloquantes pour l'apprentissage sont contournées par des approximations : un **STE** pour l'argmin de la recherche de bloc, et un bruit uniforme additif pour remplacer l'arrondi de la quantification. Le gain moyen rapporté est d'environ 11% sur plusieurs codecs (**AVC**, **AV1**, **VVC**).

Limites. L'implémentation paraît simple, mais cette simplicité vient surtout de la difficulté à reproduire fidèlement un codec complexe tout en gardant des fonctions dérivables et compatibles avec la mémoire des cartes graphiques (la recherche de blocs est notamment coûteuse en mémoire). Par

ailleurs, l'estimation du débit ne repose pas sur un calcul figé : elle s'appuie sur un modèle d'entropie appris, ce qui réintroduit une composante apprise là où l'on cherchait justement à s'affranchir de composantes apprises par IA et à reproduire la logique du codage théorique classique.

Sandwiched Video Compression (Google, ICIP 2023) — codec simplifié avec encadrement pré/post [15]

Cette approche reprend l'idée du « sandwich » ou similaire au « neural wrapper » vu avant : un filtre de pré-traitement et un filtre de post-traitement, très légers, encadrent un **codec** standard. Le proxy de codec est, comme pour l'exemple précédent, une version optimisable simplifiée : codage intra de type image (**DCT** et quantification avec un estimateur de type *straight-through*, **STE**), compensation de mouvement, sélection de mode inter/intra, codage du résidu, filtre de boucle, et une estimation de débit différentiable et simple. Cette approche obtient dans certains cas des gains aux alentours de 30 % de débit économisé sous la métrique perceptuelle **LPIPS**.

Limites. L'évaluation porte sur **H.265** et sur des scénarios assez spécifiques, principalement avec le **PSNR** et **LPIPS**, ce qui rend la transposition directe à notre cas moins évidente. Et là encore, le post-traitement côté client pose les mêmes difficultés de déploiement. On peut d'ailleurs imaginer que les résultats sont en bonne partie dus à ce filtre de post-traitement, comme pour le premier papier présenté.

5. Implémentation et Méthode

5.1. Objectif

Comme évoqué dans la partie précédente, de nombreux travaux sont parvenus à dépasser les limites des codecs classiques pour entraîner un filtre. Mais la diversité des implémentations, des méthodes d'entraînement et d'évaluation rend difficile de savoir laquelle est la plus efficace dans notre cas précis : le filtre de prétraitement. À cela s'ajoute un obstacle pratique, ces travaux sont rarement accompagnés de leur code source, ce qui complique fortement leur ré-implémentation et, bien souvent, empêche d'en reproduire les résultats.

Il était donc indispensable de reprendre ces idées, de les adapter et d'en évaluer la pertinence, afin d'identifier les outils les plus adaptés à notre projet. Une contrainte nous était par ailleurs propre, notre cible était **H.265**, ce qui n'était pas forcément le cas des travaux cités.

Concrètement, notre implémentation poursuivait deux objectifs : reproduire un proxy par codage neuronal adapté à **H.265**, et réaliser une version simplifiée d'un **codec** cherchant à se rapprocher de sa logique de compression.

5.2. Architecture globale

Afin de comprendre les différents éléments nécessaires à l'apprentissage du filtre, voici un schéma qui explique la logique globale de l'apprentissage pour notre cas d'utilisation, une logique qui s'appliquera aux deux implémentations testées.

Dans ce schéma, le terme **proxy** est associé à la copie du codec pour simplifier la compréhension, les scores liés aux **métriques** sont aussi remplacés par **résultat du filtre**, car dans cette boucle, les métriques nous servent de guide et définissent les résultats du filtre durant l'apprentissage.

Il est important de différencier le proxy du véritable codec. Pour simplifier, le filtre va agir sur les images qui seront ensuite encodées et décodées par le vrai codec, puis par son proxy, les métriques évalueront les images produites par le véritable codec mais n'étant pas différentiable (pas de pente partout), l'optimisation se fera sur le proxy. Cela permet de s'assurer que le comportement du filtre fonctionne sur le véritable outil cible (**H.265**), on contourne alors les limitations grâce à cette méthodologie.

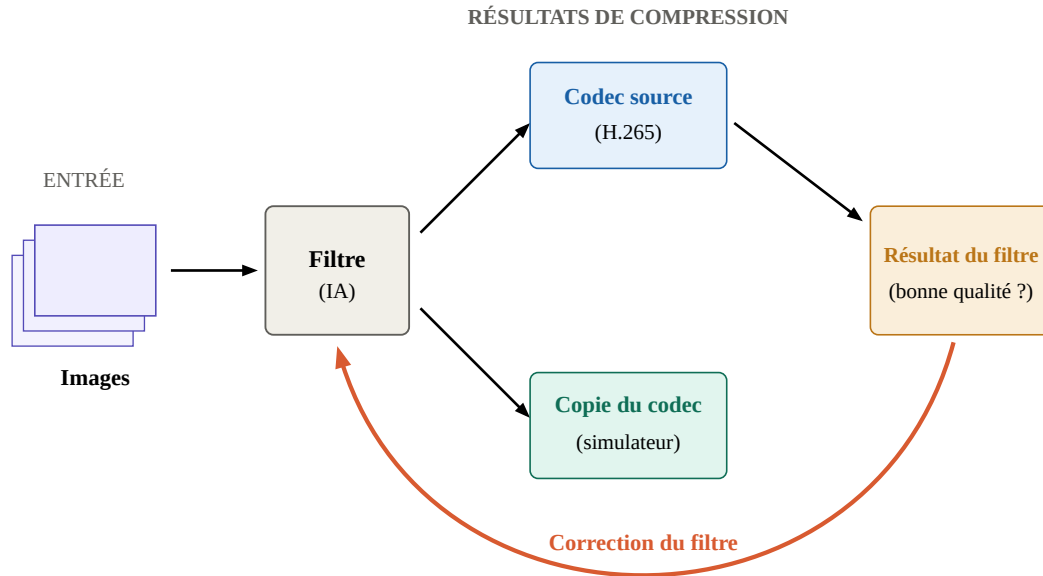


Fig. 28. – Déroulement de la boucle d'apprentissage du filtre

Pour entrer plus en détail concernant l'architecture du filtre utilisée, nous avons fait le choix de reprendre une architecture simple liée à la partie pré-filtrage du papier [13] présenté plus tôt. Le détail des éléments composant ce filtre est disponible en Annexe B. Les détails de l'architecture étant fournis, cela facilite grandement l'implémentation. L'objectif ici est d'évaluer les solutions de proxy avec une architecture de filtre fonctionnelle, cette architecture étant validée par les résultats fournis dans ces travaux, la reprendre simplifie l'étude de ce point. Dans une perspective future d'évolution visant des gains supplémentaires, cette architecture pourra largement évoluer. Par exemple elle ne travaille que sur une image à la fois et uniquement sur la luminance, ce qui limite peut-être son champ d'action, d'autant plus qu'elle était prévue pour fonctionner avec un filtre de post-traitement.

5.3. Proxy par codage neuronal

Cette première implémentation s'appuie sur la littérature récente et repose sur un **codec neuronal**. L'intérêt d'un tel outil est qu'il est, contrairement à **H.265**, entièrement différentiable : il peut donc prendre la place du **codec** au cœur de la boucle d'apprentissage, à condition d'être au préalable amené à se comporter comme lui.

Pour comprendre la suite, on peut décrire un codec neuronal de façon simplifiée comme deux branches qui communiquent. La première encode puis décode l'information, elle transforme l'image en représentations abstraites, plus légères à transmettre, avant de reconstruire l'image à partir de ce qui a été reçu. La seconde estime le poids de cette information, c'est-à-dire en quelque sorte la complexité de l'image, on retrouve ici un lien direct avec la théorie de l'information présentée en Chapitre 4.4. Cette séparation est importante, c'est elle qui permettra d'aligner séparément la qualité et le débit sur le codec cible en d'autres termes dire si l'effet du filtre permet de réduire le coût de l'image.

Ce fonctionnement a deux conséquences. D'abord, ces deux branches doivent être entraînées avec des objectifs différents : l'une sur la qualité de reconstruction, l'autre sur l'estimation du débit. Ensuite, et c'est le point le plus important, le codec encode à une qualité fixe, il n'existe pas de paramètre permettant de lui demander de compresser plus ou moins fortement, il code à la qualité qu'il a apprise. Si l'on veut couvrir plusieurs niveaux de qualité, il faut donc entraîner plusieurs modèles. Comme évoqué plus tôt, des approches récentes lèvent en partie cette limite en introduisant un débit variable, par exemple par modulation des caractéristiques [16], mais elles restent plus délicates à réentraîner pour notre cas d'usage, notamment car leur code source ne le permet pas.

Ces différents aspects se traduisent par plusieurs points de contrôle dans la construction de la pipeline d'entraînement. Il faut d'abord faire du codec neuronal un bon imitateur de notre codec cible, en deux temps : on l'entraîne d'abord à reproduire visuellement les images produites par **H.265** (avec ses défauts), puis on aligne son estimation de débit pour qu'il renvoie un coût proche de celui de **H.265**. Ce pré-entraînement en deux étapes est relativement coûteux.

Vient ensuite l'entraînement du filtre lui-même. Là, une difficulté apparaît : le filtre modifie les images, or le codec neuronal a appris à imiter **H.265** sur des images « normales ». Il risque donc de mal se comporter face à ces images modifiées, et de dériver. Pour limiter ce problème, nous avons retenu deux précautions. La première est de faire en sorte que le filtre se comporte au départ comme une fonction neutre, c'est-à-dire qu'il reproduise quasiment son image d'entrée, cela évite d'être confronté d'emblée à des images trop différentes issues d'un filtre encore aléatoire. La seconde, reprise de la méthodologie de [13], consiste à réaligner régulièrement le codec sur les images filtrées pour compenser cette dérive. Nous avons toutefois fait le choix de réaligner le codec à chaque epoch (étape d'apprentissage), c'est-à-dire au bout d'un certain nombre d'itérations plutôt qu'à chaque itération, l'entraînement s'avérant plus stable et plus rapide de cette manière.

Reste la question de la qualité, puisque le codec encode à un niveau fixe. La logique du projet étant de se rapprocher des standards de la **VO**, nous avons choisi de travailler à un **CRF** (*Constant Rate Factor*) de 22, un bon compromis. À ce niveau, le filtre est entraîné sur une qualité élevée, il ne peut donc pas détruire massivement l'image, puisqu'il doit préserver cette qualité tout en réduisant le débit. On peut alors espérer que son comportement reste intéressant à plus basse qualité. Un entraînement à basse qualité aurait pu se justifier si la cible était explicitement les vidéos peu compressées, mais le filtre risquerait alors d'apprendre à sacrifier certains détails pour mieux préserver l'image globalement, un comportement qui ne serait probablement pas apprécié à haute qualité.

5.4. Proxy par codec simplifié

Afin de développer une méthode qui prend davantage en compte les réalités de la compression, nous avons fait le choix de créer une seconde version de proxy, cherchant cette fois à devenir un simulateur fidèle de **H.265**. La difficulté principale réside dans le fait que de nombreuses opérations complexes, optimisées pour s'exécuter rapidement sur des processeurs, doivent ici être transformées

en simplifications à la fois différentiables et exécutables sur carte graphique. Cette dernière possède des propriétés très différentes du CPU, mais reste indispensable à l'entraînement par IA.

Comme nous l'avons vu dans le Chapitre 4.7.1, les codecs comportent un grand nombre d'opérations inutilisables en l'état pour l'apprentissage, faute d'une pente exploitable. Cette implémentation repose donc sur un compromis : trouver des fonctions de remplacement qui restent fidèles à la logique des opérations principales du codec. Une telle version perd en fidélité, mais conserve l'essentiel, en restant dans le même paradigme que la cible **H.265**, là où un proxy purement neuronal s'en éloigne.

5.4.1. Sélection du meilleur bloc : du choix dur au mélange pondéré

La sélection du meilleur bloc candidat lors de la prédiction est l'une des opérations bloquantes identifiées précédemment, elle repose sur un **argmin**, un choix discret qui ne fournit aucun moyen d'optimiser comme présenté dans la figure (Fig. 26). Les travaux existants comme [14] conservent cette opération mais la contournent par un **STE**, forçant un gradient identité au retour (il ne change pas avec cette étape). Cette approche fonctionne, mais elle laisse passer l'opération, elle ne porte alors d'information que sur le bloc effectivement choisi, ignorant les autres candidats.

Nous avons aussi retenu une autre approche, plutôt que de sélectionner un unique bloc, nous calculons une combinaison pondérée de l'ensemble des candidats. Chaque candidat reçoit un poids d'autant plus fort que son erreur de correspondance est faible. Concrètement, la prédiction finale mélange visuellement plusieurs blocs selon leur pertinence. L'intérêt est que le gradient circule à travers tous les candidats, et plus uniquement le gagnant.

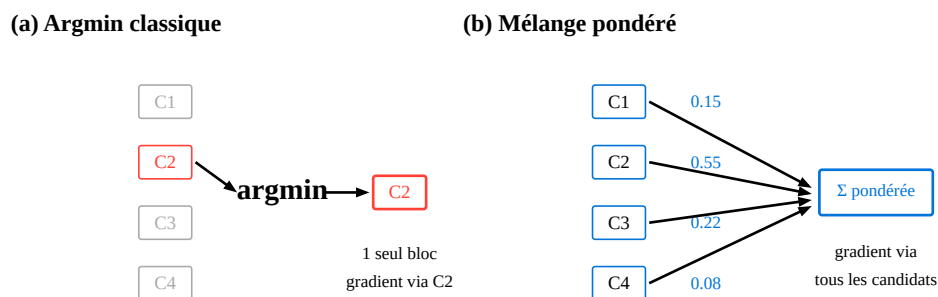


Fig. 29. – Deux façons de gérer la sélection de bloc non différentiable. À gauche, le **STE** garde le choix dur (**argmin**) et ne propage le gradient que par le bloc gagnant. À droite, notre approche par mélange pondéré (**softmax**) combine tous les candidats selon leur pertinence, laissant le gradient circuler à travers chacun.

5.4.2. Zone de recherche et prédiction

Pour rester exécutable sur carte graphique malgré le coût des recherches de blocs, [14] propose une méthodologie reproduisant la complexité des opérations de prédictions pour les images clés, tout en respectant les contraintes mémoire car ce sont des opérations qui sont facilement exécutables sur des cartes graphiques. Nous reprenons alors une logique qui ressemble à la prédiction inter (Fig. 13), mais en l'appliquant ici à l'intérieur d'une seule image, la zone de recherche y est restreinte à ce qui est théoriquement déjà connu dans l'image en cours de décodage, c'est-à-dire la partie située au-dessus du bloc courant et à sa gauche (Fig. 30). Cette contrainte est essentielle, un décodeur ne dispose jamais des blocs « futurs », et l'ignorer reviendrait à entraîner le filtre sur une information indisponible à la reconstruction réelle. Cette méthode permet alors de reproduire des transformations complexes pour un coût minime, il serait presque impossible de réaliser des opérations classiques de manière conditionnelle comme le ferait un vrai codec et ces opérations non optimisables mises bout à bout rendraient le terrain d'apprentissage très pauvre. Cette méthode semble donc être un bon compromis entre faisabilité et respect de la complexité des méthodes d'origine. Pour résumé on va donc chercher le meilleur bloc dans une zone restreinte, sans ce limiter à des opérations conditionnelles comme une prédiction d'image clé classique. La prédiction des images suivantes se fait d'une manière similaire, mais en prenant en compte l'image précédente pour la zone de recherche (on reste ici très proche du fonctionnement classique).

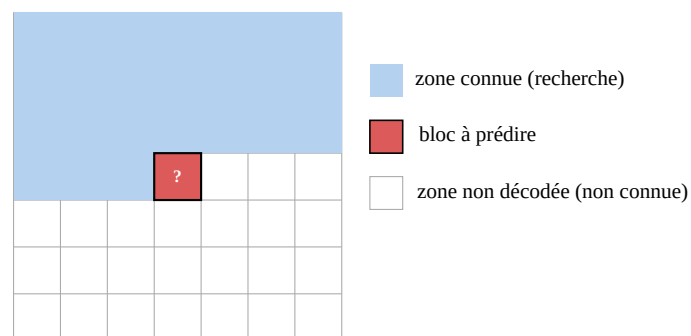


Fig. 30. – Zone de recherche causale pour la prédiction d'un bloc. Seule l'information déjà décodée (au-dessus et à gauche) est disponible.

5.4.3. Quantification et transformée

Une autre étape clé identifiée précédemment est la **quantification**, dont l'arrondi supprime de l'information et présente un gradient nul presque partout (Fig. 26). Plusieurs options existent pour la rendre apprenable : remplacer la fonction pour l'apprentissage des poids par une alternative d'arrondi modifiée et optimisable en gardant la fonction de base pour le reste des opérations.

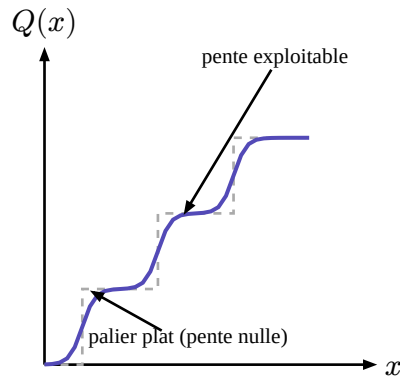


Fig. 31. – Arrondi dur (escalier, en pointillés) et sa version adoucie continue (*arrondi doux*).

L'arrondi réel a un gradient nul presque partout, la version adoucie suit les mêmes marches tout en gardant une légère pente, ce qui rend cette fonction optimisable.

L'autre option est de simuler la perte causée par cet arrondi par une alternative différente, par l'ajout d'un bruit aléatoire, comme dans les travaux de [14]. L'idée est de remplacer l'arrondi dur, non différentiable, par une perturbation dont l'effet s'en approche, tout en étant une fonction optimisable.

Concernant la transformation de l'image, les calculs reposent sur des matrices de **DCT** permettant un traitement rapide. Notre cible étant **H.265**, nous avons veillé à employer les matrices effectivement utilisées par ce codec, et non les matrices théoriques de la **DCT**. C'est une différence notable avec les différents projets, des travaux comme [14] visaient une optimisation plus générale, sans s'aligner sur un codec précis, là où notre objectif est explicitement de coller au mieux au comportement de **H.265**.

Enfin, nous avons fait le choix de travailler sur des blocs de taille 8×8 . Une évolution future pourra consister à introduire des tailles de blocs variables, comme le fait un codec réel, puis à en valider l'utilité dans notre cas d'usage.

5.4.4. Estimation du coût des images

Dans cette implémentation, il est aussi important d'établir la manière dont on va permettre à notre outil d'estimer le coût d'une image. Nous l'avons vu, le papier [14] utilise une méthode apprise (IA), qui semble fonctionner mais la problématique de ce genre d'approche est la généralisation face aux différentes qualités possibles.

Une autre méthode est proposée dans les travaux [15] : une fonction simple, non apprise, qui reprend directement les principes présentés en Chapitre 4.4.

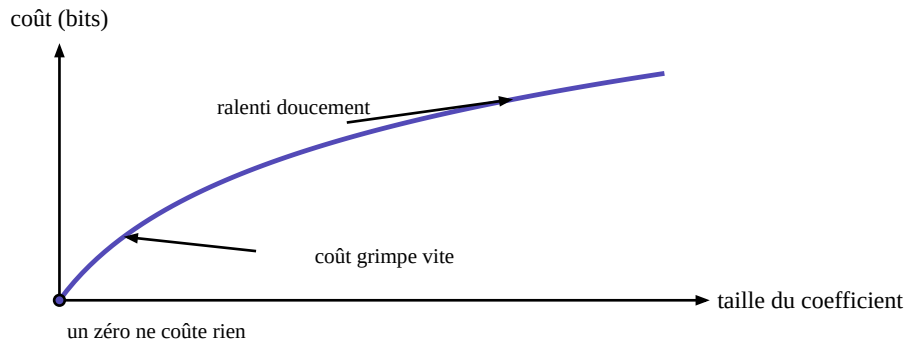


Fig. 32. – Comportement de la fonction d'estimation du coût [15] : le coût d'un coefficient monte franchement au début, puis se courbe de plus en plus doucement. Un coefficient nul ne coûte rien.

L'idée découle de la théorie de l'information, le coût d'un symbole ne dépend pas de sa valeur, mais de sa probabilité, plus un symbole est probable, moins il coûte cher à transmettre. Or, dans un codec, la quantification ramène une grande partie des coefficients vers zéro (voir Chapitre 4.4). Les petites valeurs deviennent donc très fréquentes, et les grandes, rares. C'est ce qui autorise un raccourci utilisé ici, plutôt que d'estimer la probabilité de chaque coefficient, on relie directement son coût à sa taille. Un coefficient proche de zéro, fréquent, est quasiment gratuit à transmettre, tandis qu'un grand coefficient, rare, coûte cher.

Cette forme, par sa pente, pousse le filtre à réduire l'amplitude des coefficients en particulier quand ils sont déjà proches de 0 (où la pente est plus forte). C'est exactement l'objectif recherché, simplifier l'image pour la rendre moins coûteuse à coder.

Ce choix est fait car obtenir la probabilité d'un coefficient est difficile, encore plus de l'obtenir en répliquant les méthodes d'un véritable codec et rendre ces étapes complexes optimisables, devient alors presque impossible. Cette simplification, imparfaite permet tout de même d'obtenir une estimation proche et reste simple à optimiser.

5.4.5. Essais et échec d'implémentation

Durant le projet de nombreux tests ont été réalisés, beaucoup n'ont pas permis d'aboutir à une solution fiable mais ce sont aussi ces tests qui ont permis de continuer à rentrer plus en détail dans le sujet. Il existe un grand nombre de possibilités et établir les meilleures options est un problème complexe qui demande aussi une connaissance dans le domaine très poussée. Parfois certaines tentatives sont aussi des pertes de temps sèches, j'ai fini par les éviter le plus possible en me fixant des limites et en repartant parfois de choses plus simples mais établies afin de ne pas me perdre dans des solutions inutiles.

Il est important aussi de préciser que certains tests n'ont pas réellement été des échecs mais la temporalité liée au projet de fin d'études limite le champ d'action, certaines idées ou améliorations prendront alors place pour le futur du projet selon leur pertinence.

5.4.6. Bilan de l'implémentation

Cette implémentation s'est révélée particulièrement intéressante pour tester différents mécanismes et mettre en relation la théorie évoquée dans le document avec des cas d'utilisation concrets. Durant son développement, plusieurs essais rapides n'ont apporté aucune amélioration nette, voire ont entraîné de fortes pertes, ce qui a permis de converger vers les choix présentés ci-dessus. L'objectif étant précisément de définir les bonnes méthodes pour fiabiliser un tel outil, des études comparant ces différentes méthodes sont présentées en Chapitre 6.

5.5. Guide d'optimisation

Nous l'avons vu, le guide d'apprentissage doit satisfaire deux exigences à la fois : être un bon simulateur de ce que percevrait un utilisateur, et rester facilement optimisable pour que les poids du filtre soient ajustés dans la bonne direction.

De nombreuses combinaisons de **métriques** sont possibles, mais toutes ne peuvent pas être testées. Or, l'objectif à cette étape du projet porte avant tout sur les différentes approches de remplacement du **codec** durant l'apprentissage, et non sur la recherche du guide idéal, une analyse qui est d'ailleurs réalisée par d'autres membres de l'équipe. Il paraît donc justifié de retenir un choix simple et cohérent, respectant nos critères de départ, quitte à approfondir cet aspect dans la suite du projet.

À cela s'ajoute une précaution méthodologique, nous l'avons évoqué dans les limites des travaux existants, certains projets entraînent le filtre sur une métrique puis évaluent les performances sur cette même métrique, ce qui fausse probablement l'analyse. Nous chercherons donc à dissocier le guide d'apprentissage des outils servant à l'évaluation finale.

Le guide retenu cherche un compromis entre simplicité et fidélité à la perception humaine. Il associe deux composantes : une perte L1 et la métrique **DISTS**.

La perte L1 est une fonction simple, orientée fidélité : elle mesure l'écart direct entre l'image filtrée et l'image source, dans le même esprit que le **PSNR** évoqué plus tôt, peu précis perceptuellement mais facile à optimiser. Elle joue ici le rôle de garde-fou, en empêchant le filtre de trop s'éloigner de l'image d'origine un rôle essentiel pour éviter des dérives venant d'une métrique plus complexe.

DISTS apporte la dimension perceptuelle. Elle s'appuie sur un réseau de neurones (IA) et est calibrée sur des jugements humains de similarité. Entraînée à partir de paires d'images, elle est théoriquement capable de savoir si une texture différente au niveau des pixels est acceptable pour un utilisateur. Ce qui rend **DISTS** pertinent ici est donc sa tolérance aux textures. Autrement dit, elle laisse au filtre la liberté de modifier l'image au niveau du détail, sans le pénaliser tant que la satisfaction de l'utilisateur n'est pas affectée. C'est exactement le type de souplesse que l'on recherche pour réduire le débit sans dégrader la qualité perçue. De plus elle a été conçue pour ce type de projet de modification d'images.

L1 et **DISTS** sont donc complémentaires : la première ancre le résultat sur la source et évite les dérives, la seconde guide les modifications dans une direction compatible avec la perception humaine.

Concernant l'utilisation d'autres méthodes, différents tests sont en cours mais ne sont pas à un stade assez abouti pour les évoquer dans ce document.

5.6. Méthode d'évaluation

La méthode d'évaluation est un point essentiel du projet. Nous l'avons vu (Chapitre 4.6), il existe de nombreux outils prévus pour évaluer la qualité vidéo. L'objectif pour nous est de définir les meilleurs outils pour refléter l'avis d'un utilisateur.

Dans notre cellule de nombreuses analyses ont été réalisées sur les **métriques** afin d'estimer quels outils pourraient être les plus pertinents pour ce projet. Nous verrons dans cette section un bref résumé qui expliquera alors les choix réalisés.

On compare donc plusieurs **métriques** selon leur accord avec le jugement d'utilisateurs (le score **MOS**). Plus le score (corrélation de Spearman) est proche de 1, plus la métrique classe les vidéos comme le ferait un observateur humain. On teste cet accord dans deux situations : quand le contenu varie peu dans l'image (faible variabilité) et quand il varie beaucoup (forte variabilité).

Pour comprendre les résultats il est aussi important de présenter les différentes métriques.

VMAF (Video Multi-Method Assessment Fusion) est une métrique que nous avons déjà évoquée, elle possède aussi une version durcie, VMAF-NEG, qui pénalise un rehaussement du contraste, elle est reconnue pour être plus robuste à certaines conditions grâce à des limites que la version classique n'a pas. UVQ (Universal Video Quality) est une métrique plus récente, basée sur l'IA, qui a appris à partir de scores **MOS**. **LPIPS** (Learned Perceptual Image Patch Similarity) est une métrique basée sur un réseau de neurones entraîné pour prédire la similarité perceptuelle entre deux images.

Les tableaux ci-dessous ne montrent que les métriques retenues comme candidates, les autres, moins pertinentes pour notre usage, ont été écartées pour la clarté.

Les deux tests suivants sont ciblés. Le premier jeu de données est basé sur des vidéos qui ont été encodées avec différentes versions du codec **H.265**, qui est notre cible d'optimisation, et des variations de qualité. Avoir une métrique qui réagit de manière pertinente face aux vidéos qui sortent de ces différentes versions est un bon indicateur de sa pertinence pour notre projet.

Le second jeu de données est basé sur des vidéos encodées avec un encodage par régions d'intérêt (ROI), c'est-à-dire que certaines zones de l'image sont encodées avec plus de qualité que d'autres, ce qui est un cas proche d'un filtrage IA. Avoir une métrique qui réagit de manière pertinente face à ces vidéos est aussi un bon indicateur de son utilité pour notre projet.

Métrique	Faible variabilité ↑	Forte variabilité ↑
VMAF	0.926	0.737
VMAF-NEG	0.928	0.721
UVQ	0.756	0.824
Delta UVQ	0.937	0.722
LPIPS	0.786	0.520
CVVDP	0.782	0.451

Tableau 2. – Accord avec la référence sur différents encodages **H.265**. ↑ : plus haut = meilleur.

Métrique	Faible variabilité ↑	Forte variabilité ↑
VMAF	0.899	0.808
VMAF-NEG	0.803	0.702
UVQ	0.803	0.754
Delta UVQ	0.904	0.850
LPIPS	0.881	0.880
CVVDP	0.905	0.784

Tableau 3. – Accord avec la référence sur un encodage par régions d'intérêt (ROI), un cas proche d'un filtrage IA. ↑ : plus haut = meilleur.

VMAF et UVQ sont les valeurs sûres. Elles restent bien classées dans presque toutes les situations : VMAF est solide un peu partout, et UVQ atteint souvent de bonnes performances.

Un point mérite d'être précisé au sujet d'UVQ : c'est une métrique *No-Reference*, c'est-à-dire qu'elle évalue une vidéo à partir de sa seule image, sans la comparer à l'image d'origine. Cela pose un problème dans notre cas : deux vidéos issues d'une même source peuvent avoir des qualités différentes, qu'UVQ jugera sans tenir compte de ce point de départ commun. C'est là qu'intervient la variante **Delta UVQ**, qui corrige le score en le ramenant à celui de la vidéo d'origine, rétablissant ainsi une comparaison équitable entre les contenus. On voit dans les tableaux que ce recalage améliore nettement les scores (0.937 contre 0.756 en faible variabilité sur le premier jeu, 0.850 contre 0.754 en forte variabilité sur le second). Dans notre cas d'usage, où l'on compare deux versions d'une même vidéo face à une référence commune, ce recalage est moins déterminant, mais il montre qu'UVQ peut être exploitée différemment pour gagner en fiabilité et montre surtout sa pertinence face à un contenu avec une référence égale.

Ce sont donc des candidats assez fiables et polyvalents pour évaluer notre filtre. Elles sont d'ailleurs reconnues dans la littérature pour leur fiabilité, et sont utilisées dans de nombreux travaux et aussi par l'industrie.

LPIPS devient intéressante dans notre cas précis. Le second jeu de données repose sur un encodage par régions d'intérêt, c'est-à-dire un codage qui concentre ses efforts sur les zones importantes de l'image, exactement le genre de comportement qu'un filtre IA cherche à produire. Or c'est justement là que LPIPS obtient ses meilleurs scores (0.881 et 0.880, la meilleure métrique en forte variabilité). Comme ce cas d'usage ressemble au nôtre, LPIPS mérite d'être considérée, alors qu'elle était bien moins convaincante sur l'encodage classique.

CVVDP montre aussi de belles performances face au contenu ROI. Sa conception, très orientée pour reproduire la vision humaine, semble aussi être un argument supplémentaire qui pousse à prendre en compte cette mesure pour l'étude des résultats.

En résumé, **VMAF** et **UVQ** s'imposent comme les métriques principales, tandis que LPIPS pourrait apporter des informations complémentaires par sa pertinence face à un cas d'usage proche.

Nous le voyons ici, aucune métrique n'est parfaite face à la perception humaine, il est donc important de croiser les résultats pour avoir une idée plus précise de la qualité des images reconstruites. On peut cependant évoquer la limite de ces métriques pour notre cas d'utilisation, des images modifiées par IA, où il est difficile pour le moment d'assurer la fiabilité des résultats obtenus. Ce qui mènera ce projet vers des tests face à de vrais utilisateurs dans le futur.

5.7. Environnement de test

Notre cible étant des vidéos de VOD il est important d'utiliser les outils de manière à refléter ce qu'on pourrait retrouver en situation réelle. Le choix a donc été fait de tester les solutions face à une version optimisée de H.265. Il existe différents modes qui permettent d'encoder et décoder plus ou moins rapidement, ce qui va aussi jouer sur la qualité des vidéos finale mais aussi sur le poids des vidéos.

Ce choix de test semble important pour vérifier la validité d'un tel filtre pour notre cible. Mais par des contraintes liées à la durée d'entraînement, la version utilisée lors de l'apprentissage sera, elle, plus rapide (moins optimisée) car cette différence de temps de traitement devient alors trop importante au vu du nombre de vidéos à traiter durant l'apprentissage.

Ce point sera aussi à étudier par la suite car cela entraîne un écart entre le domaine d'entraînement et le domaine de test.

5.8. Données d'entraînement

Pour entraîner un réseau de neurones, les données utilisées sont un point clé : il faut disposer d'un grand nombre d'exemples, mais aussi d'exemples de bonne qualité. Il existe assez peu de jeux de données qui répondent à ces exigences tout en étant accessibles publiquement. Nous avons fait le choix de reprendre un dataset utilisé dans de nombreux projets liant l'image et le deep learning,

notamment pour les travaux de [13], le dataset Vimeo90K, qui possède un grand nombre d'images (environ 90 000). La qualité des images étant variable, on peut toutefois considérer ce point comme un défaut pour notre cas d'usage, qui vise des vidéos de bonne qualité pour de la diffusion VOD.

5.9. Limites et perspectives

Une première limite tient au jeu de données. Nous avons repris celui du papier [13], ce qui présentait l'avantage de s'appuyer sur une base déjà éprouvée pour ce type de travaux. Une évolution possible consisterait à passer à un jeu de données comme *BVI-DVC* comportant moins d'images, mais composé d'images de meilleure qualité et plus récentes. Il serait alors possible de remplacer ou de combiner les deux jeux de données pour obtenir plus de données, plusieurs options s'offrent alors. Un tel changement pourrait améliorer la pertinence de l'apprentissage, et fait partie des pistes envisagées pour la suite du projet.

La seconde limite concerne le guide d'apprentissage. Le choix retenu ici est volontairement simple, et il gagnera à être optimisé pour obtenir des résultats plus nets. Ce point n'est pas un oubli mais une étape à part entière du projet, l'objectif de ce PFE portait avant tout sur les approches de remplacement du codec, et l'affinement du guide constitue un travail ultérieur, que les outils mis en place ici rendront justement possible par la suite.

6. Résultats et analyses

6.1. En tant que Proxy

Cette section a pour but d'évaluer les performances de ces différentes méthodes en tant que copie de ce que fait réellement **H.265**, le but est donc d'évaluer la pertinence des images sorties par ces outils, mais aussi d'évaluer si elles sont capables d'ordonner de manière précise les différents contenus au niveau du débit et donc du coût réel des différentes vidéos.

Cette première évaluation permet aussi de mettre en avant les différentes approches pour la version du proxy simplifié, et voir et comprendre les mécanismes qui vont jouer ou non sur la qualité de reconstruction. Ce qui permettra de réaliser des choix plus pertinents pour la suite du projet.

Les deux méthodes étant différentes, cette évaluation se fera donc sur un point de qualité qui peut être atteignable par les différentes options. Pour rappel, le proxy neuronal est entraîné à reproduire une qualité fixe (**CRF 22** dans notre cas), pour le comparer avec la seconde approche il faut alors se placer à **QP 22**, ce qui va légèrement différer mais permet tout de même de voir à quel point ces outils sont pertinents ou non, pour en valider les résultats, nous verrons aussi les résultats à **CRF 22**.

Ces tests seront réalisés sur des vidéos du dataset MCL-JCV, qui est un dataset de référence pour l'évaluation de la compression vidéo. L'objectif est de ne pas utiliser des vidéos similaires à celles qui ont permis d'entraîner le proxy neuronal, afin de ne pas biaiser les résultats. Le dataset MCL-JCV est composé de 30 vidéos HD, ce qui suffit pour obtenir des résultats fiables. Il est aussi intéressant de noter que ce dataset est composé de vidéos très différentes, ce qui permet d'avoir une idée plus précise de la pertinence des outils sur différents types de contenus.

6.2. Fidélité de reconstruction

Pour évaluer la fidélité de structure des images, nous l'avons vu, **PSNR** reste une métrique simple, peu fiable pour la perception humaine, mais qui est un bon indicateur de la fidélité de reconstruction. Ici l'intérêt est d'estimer les performances des outils pour reproduire ce que fait le vrai codec, l'utiliser reste donc un choix simple et pertinent. **SSIM** est lui plus précis et plus proche de la perception humaine et évalue la similarité structurelle. Il est donc intéressant de le croiser avec le PSNR pour avoir une idée plus précise de la qualité des images reconstruites. De plus durant l'entraînement du proxy neuronal, la métrique MSE (Mean Squared Error) est utilisée pour guider l'apprentissage, cette métrique étant directement liée à la logique du PSNR, il est donc logique de la croiser avec le SSIM pour avoir une idée plus précise de la qualité des images reconstruites, sans biais.

Pour rappel plus un score PSNR est haut, plus cette mesure considère l'image comme fidèle à l'originale, aux alentours de 40 une image est considérée comme très fidèle. Concernant SSIM les scores varient entre 0 et 1, plus on se rapproche de 1, plus l'image est considérée comme similaire.

On sépare ici les images intra des images inter car nous utilisons des méthodes qui diffèrent de celles d'un codec classique, cela permet de valider ou d'invalider ces deux étapes.

	PSNR intra ↑	SSIM intra ↑	PSNR inter ↑	SSIM inter ↑
Arrondi · softmax	40.41	0.9574	39.52	0.9507
Arrondi · argmin	39.43	0.9453	38.35	0.9319
Bruit · softmax	37.82	0.9184	37.22	0.9110
Bruit · argmin	37.83	0.9184	37.22	0.9110
Neuronal	36.85	0.9769	40.68	0.9805

Tableau 4. – Fidélité proxy vs H.265, mode CRF. ↑ : plus haut = meilleur.

	PSNR intra ↑	SSIM intra ↑	PSNR inter ↑	SSIM inter ↑
Arrondi · softmax	42.70	0.9738	41.57	0.9664
Arrondi · argmin	41.41	0.9627	40.00	0.9480
Bruit · softmax	39.40	0.9363	38.66	0.9272
Bruit · argmin	39.40	0.9363	38.66	0.9272
Neuronal	36.97	0.9756	41.17	0.9817

Tableau 5. – Fidélité proxy vs H.265, mode QP constant. ↑ : plus haut = meilleur.

Le proxy neuronal reconstruit les images les plus ressemblantes, en particulier sur les images prédites à partir des précédentes (colonnes *inter*), où il devance nettement toutes les autres versions en particulier avec le mode CRF. Ce réseau a été entraîné pour copier le vrai codec sur ce point, donc il excelle à cette tâche. Il est tout de même intéressant de voir que les scores PSNR ne sont pas toujours bons alors que le SSIM si, évaluant les écarts de structures. Ce qui signifie qu'il ne reproduit pas fidèlement les pixels mais conserve la structure de l'image, ce qui est finalement le but recherché. Le proxy neuronal est donc un bon imitateur du codec, en particulier au niveau structurel.

Parmi nos versions simplifiées, la variante *arrondi · softmax* est la plus proche du vrai codec, les autres réglages s'en éloignent un peu. Le softmax est la fonction que nous avons présentée, elle permet de mélanger les candidats de prédiction plutôt que d'en choisir un seul. Cela a pour effet de lisser légèrement le résultat, ce qui est apprécié par le PSNR notamment, ce qui explique en partie ces résultats. On peut tout de même valider la pertinence du proxy simplifié, car lui n'a pas appris à reproduire fidèlement les images.

Il est intéressant de noter que ces performances sont obtenues à cette qualité qui est haute mais devient moins bonne si l'on choisit de traiter des images de plus basse qualité, c'est ce qui a été remarqué en réalisant d'autres tests. On peut expliquer cela car le proxy simplifié ne reprend pas toutes les logiques d'optimisations complexes du véritable codec, des optimisations qui vont être bénéfiques surtout sur des images de plus basse qualité. Notre simulateur fonctionne donc très bien à haute qualité, moins à basse qualité en ce qui concerne la reproduction d'image. Ce qui montre tout de même sa limite d'action et possiblement un désalignement avec le fonctionnement réel encore important. Ce constat nous a donc obligé à utiliser cet outil avec plus de précaution et rester dans une plage limitée de qualité afin d'éviter de s'éloigner trop de l'outil d'origine.

6.3. Corrélation de débit

Pour estimer si nos outils sont de bons simulateurs, il est aussi important de voir s'ils sont capables de prédire dans la même direction que le vrai codec, c'est-à-dire si les vidéos qui sont plus lourdes pour le vrai codec sont aussi plus lourdes pour nos outils. Pour cela nous avons utilisé la corrélation de Spearman, qui est une mesure statistique permettant de mesurer la force et la direction entre deux variables. Cette mesure est adaptée à notre cas car elle ne se base pas sur les valeurs absolues mais sur l'ordre des valeurs, ce qui est exactement ce que nous voulons savoir : si les vidéos sont ordonnées de la même manière par nos outils et par le vrai codec.

	classement clips (intra) ↑	classement clips (inter) ↑
arrondi · softmax	0.93	0.67
arrondi · argmin	0.91	0.86
bruit · softmax	0.93	0.67
bruit · argmin	0.91	0.88
neuronal	0.87	0.85

Tableau 6. – Corrélation débit estimé vs H.265 (Spearman), mode CRF. ↑ : plus haut = meilleur.

	classement clips (intra) ↑	classement clips (inter) ↑
arrondi · softmax	0.98	0.72
arrondi · argmin	0.98	0.92
bruit · softmax	0.98	0.72
bruit · argmin	0.98	0.92
neuronal	0.93	0.91

Tableau 7. – Corrélation débit estimé vs H.265 (Spearman), mode QP constant. ↑ : plus haut = meilleur.

Ici, la version simplifiée qui faisait les plus belles images (softmax) n'est *pas* la meilleure pour juger du poids des fichiers. C'est le réglage plus proche du fonctionnement réel d'un codec (argmin) qui devine le mieux quelles vidéos seront lourdes ou légères, et il fait ici aussi bien que le proxy neuronal. En clair, faire une belle image et bien estimer le poids d'un fichier sont deux qualités différentes, et aucune version simplifiée ne gagne sur les deux tableaux à la fois, c'est justement la force du proxy neuronal, qui réussit bien dans les deux cas.

Concernant le point que nous avons évoqué plus tôt et qui mettait la version simplifiée en défaut face à des images encodées en basse qualité, au niveau de la corrélation, la chute de performance semble moins importante mais est tout de même présente, ce qui valide le fait de ne pas l'utiliser à des qualités trop basses.

6.4. Synthèse : deux profils, deux usages

Comme simple imitateur du vrai codec, le proxy neuronal semble le meilleur, il réalise les images les plus ressemblantes et estime bien le poids des fichiers. Rien d'étonnant, puisqu'il a été entraîné pour ça. Son inconvénient : il ne fonctionne qu'à un seul niveau de qualité, celui sur lequel il a été entraîné. On peut aussi prédire que son comportement plus éloigné du paradigme de la compression vidéo classique pourrait le rendre moins efficace si on l'utilise comme environnement pour l'apprentissage d'un filtre.

Notre proxy simplifié, lui, fonctionne comme un vrai codec, il découpe l'image en blocs et les compresse à la manière de **H.265**. Cette proximité lui donne de vrais atouts. Il estime le poids des fichiers presque aussi bien que le proxy neuronal, il reste compréhensible, et surtout il s'adapte à tous les niveaux de qualité, même s'il devient moins fidèle quand on descend en basse qualité. Ces points en font donc un bon candidat pour guider l'apprentissage d'un filtre.

De par notre implémentation présentée dans Fig. 28, les deux proxy possèdent une architecture commune qui prend la version de **H.265** en évaluation de la qualité d'image dans la boucle, et va

cependant calculer les poids sur le proxy, ce qui a aussi été expliqué dans le papier [13]. Cela permet notamment d'éviter de travailler sur un domaine trop éloigné de la cible, les images évaluées sortent alors du vrai codec. Cela signifie donc que la fidélité d'image importe moins directement dans notre architecture, car concernant la manière dont le réseau apprend, les options restent similaires (en backward), c'est le but de cette conception. Le point essentiel se trouve donc plutôt dans l'estimation de débit. Pour le proxy simplifié, les meilleurs scores sont obtenus avec la version « ste-argmin », qui est la version où l'estimation de débit reçoit bien le choix d'un bloc unique et pas un mélange de blocs mais va bien apprendre sur ce mélange de blocs. Pour la suite, fort de ce constat nous testerons alors uniquement le paramètre concernant la modification de l'arrondi, c'est-à-dire le choix entre produire un bruit aléatoire pour simuler la quantification ou réaliser cette quantification et ne pas apprendre sur cette étape. Deux paramètres notés ici « Arrondi » et « Bruit ».

6.5. Résultats face aux métriques

Dans cette section, nous verrons comment les différentes versions ont réagi durant l'apprentissage du filtre et donc les résultats obtenus grâce aux effets du filtre, ce qui permettra d'avoir une idée de la pertinence de chaque méthode.

Pour quantifier le gain d'une optimisation, on utilise le **BD-rate**. L'idée repose sur les courbes débit-distorsion : pour une vidéo, on trace la qualité obtenue en fonction du débit (le poids du fichier vidéo), à plusieurs niveaux de compression. On obtient une courbe pour la version sans filtre et d'autres pour les versions filtrées. Le BD-rate mesure alors l'écart horizontal moyen entre ces deux courbes, c'est-à-dire l'économie de débit à qualité égale. Un BD-rate de -5 % signifie que, pour une même qualité perçue, la version filtrée nécessite en moyenne 5 % de débit en moins. Plus la valeur est négative, meilleure est l'optimisation, une valeur positive indique au contraire une dégradation.

Dans un premier temps voici les courbes qui montrent visuellement les écarts entre les différentes implémentations. Il est important de noter que la métrique LPIPS est orientée différemment des autres, plus le point est bas, plus la qualité prédite est bonne, on retrouve aussi l'information sur cette direction sur chacun des graphiques.

On retrouve en bleu la version « Neuro », en rouge la version « Arrondi », en vert la version « Bruit » et en pointillés la version sans filtre.

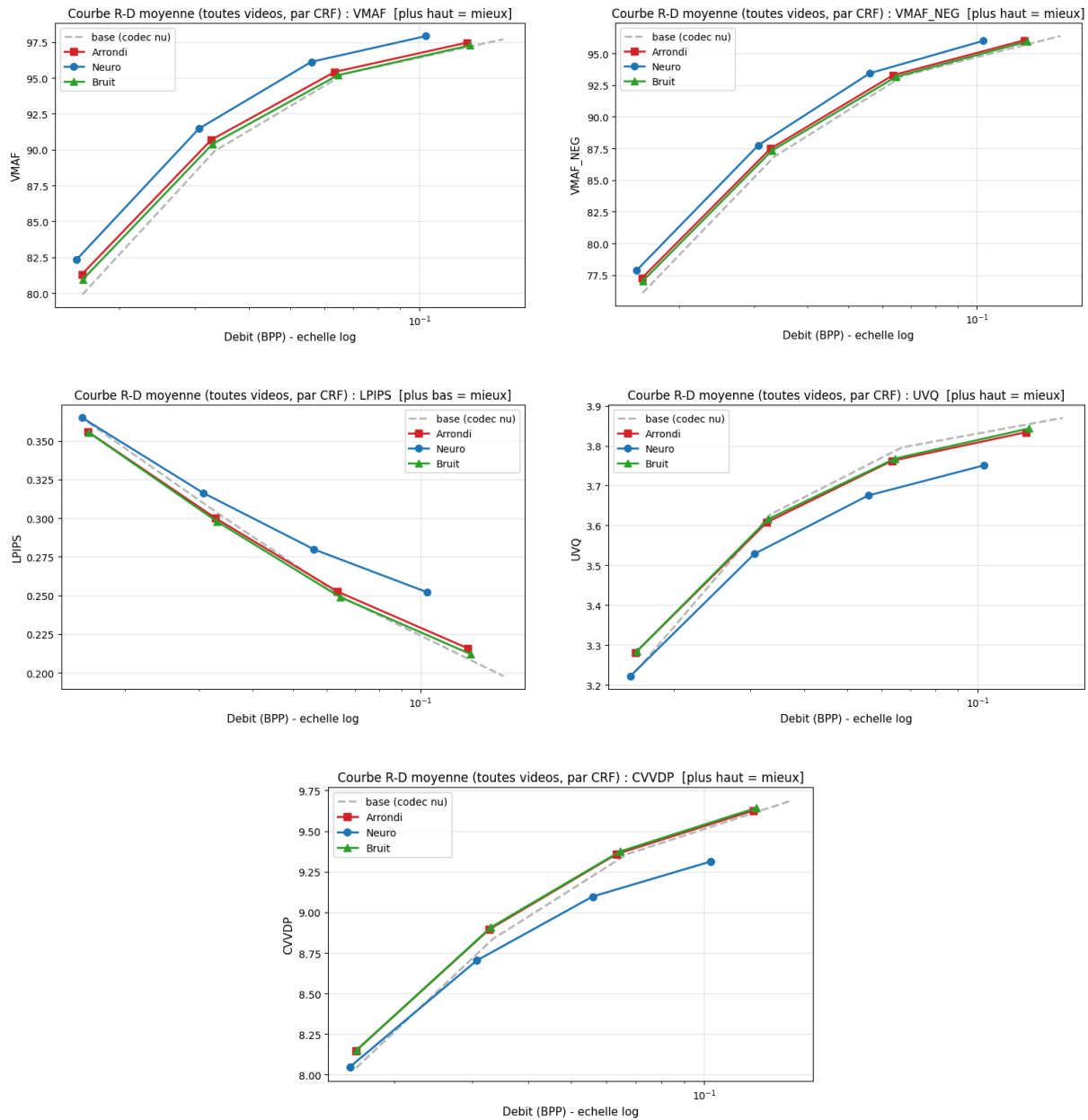


Fig. 33. – Courbes des résultats pour les métriques retenues des différentes implémentations testées

Ces courbes montrent bien une forte différence entre la version du filtre obtenu via l'entraînement sur la version neuronale et les deux versions du codec simplifié. On observe aussi une contradiction VMAF semble apprécier les résultats obtenus par la version « Neuro » alors que les autres métriques non. Concernant les deux versions du proxy simplifié on peut voir que les différences sont assez minimes.

Métrique	Bruit ↓	Arrondi ↓	Neuronal ↓
CVVDP	-7.94	-7.28	9.96
UVQ	1.61	2.88	18.13
LPIPS	-3.55	-1.15	17.62
VMAF	-4.98	-8.96	-23.33
NEG	-4.83	-7.27	-16.63

Tableau 8. – Gains de **BD-rate** (%) par métrique et par proxy. ↓ : plus bas (négatif) = meilleur.

Ce tableau reprend les informations que l'on pouvait lire sur les courbes mais de manière plus précise, ce qui permet de mettre en avant les différences entre les deux versions issues du codec simplifié. On remarque encore cette contradiction entre VMAF et le reste des métriques où la version « Arrondi » semble satisfaire majoritairement VMAF en comparaison à la version « Bruit » qui semble satisfaire les autres mesures. Cependant ces résultats montrent tout de même que la majorité des métriques semblent apprécier les effets du filtre obtenu par les version « Bruit » et « Arrondi » sur les images, en particulier à basse qualité (les points à gauche dans les graphiques précédents). Mais les contradictions entre ces différentes mesures montrent la difficulté d'analyse des résultats pour notre cas d'usage. Il semble aussi important de préciser que les différences étant faibles, on peut aussi imaginer que ces écarts se trouvent parfois dans la zone d'erreur de ces différentes mesures, ce qui remet en question ces résultats au vu de la faible différence.

Ces résultats proviennent de la moyenne des différents contenus, mais les différentes vidéos ne réagissent pas de la même manière face au filtre. Pour illustrer cela voici un exemple d'une vidéo qui semble avoir fonctionné sur les différentes mesures étudiées.

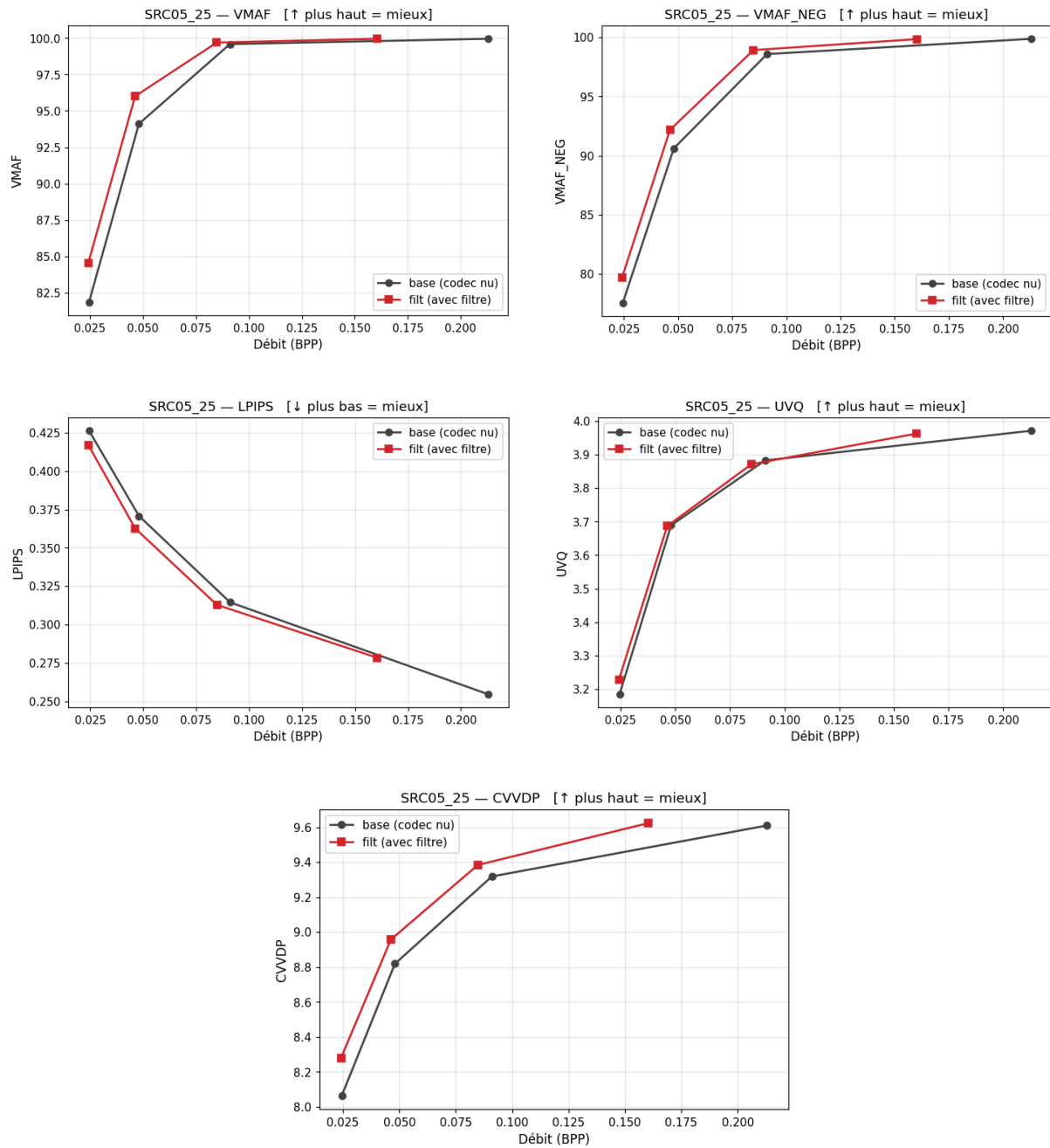


Fig. 34. – Courbes des résultats pour les métriques retenues pour la vidéo SRC05 (version « Arrondi »)

Cet exemple montre aussi que les résultats dépendent grandement du contenu, ce qui prouve l'importance des données d'entraînement, qui doivent être représentatives d'une large diversité de contenus afin d'obtenir un outil polyvalent.

6.6. Test visuel

Afin d'avoir une idée des effets du filtre sur les images voici quelques exemples où l'on voit des différences intéressantes à analyser. Ces images proviennent bien sûr des données de test et pas celles utilisées durant l'apprentissage. Il est possible que certains détails ne soient visibles qu'en zoomant. Afin d'avoir des exemples de taille suffisante, le choix a été fait de ne garder qu'un filtre parmi les deux « Bruit » et « Arrondi » comme la différence visuelle est souvent minime voire absente. La version « Arrondi » a été conservée.

Dans ces images, on retrouve l'image source, et les différentes versions ainsi que la taille de l'image en kilo-octets, une mesure informatique, plus la valeur est basse, plus la vidéo est légère.



Fig. 35. – Exemple CRF 37 (basse qualité)

On retrouve ici l'exemple où les différentes métriques se sont accordées pour un gain, on voit que le filtre a permis d'éviter d'avoir un bruit de compression important, visuellement on peut aussi confirmer ce gain.

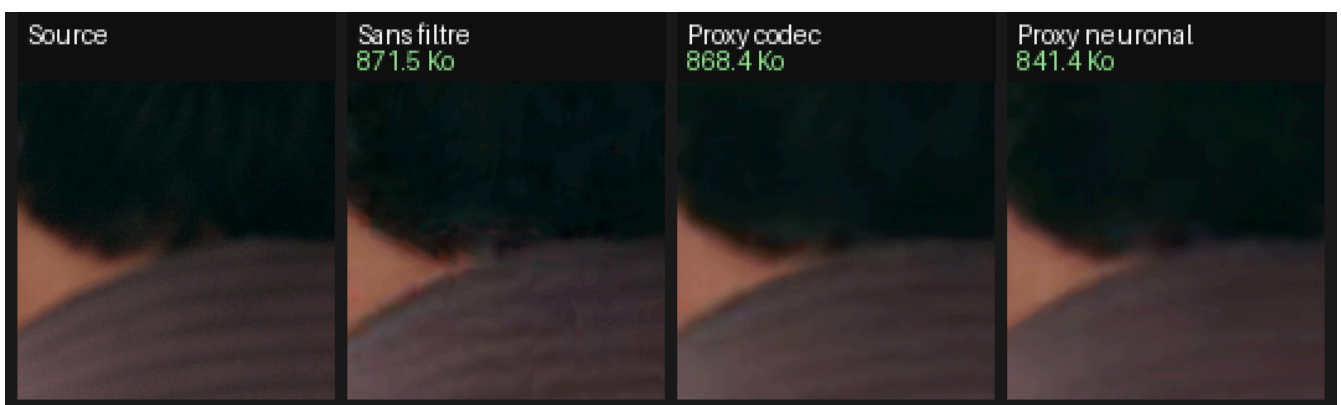


Fig. 36. – Exemple CRF 37 (basse qualité)

Cet exemple montre un comportement similaire où le filtre semble avoir fonctionné et permis de garder des textures plus naturelles. Ici non plus pas de contradiction entre les différentes mesures pour ce niveau de qualité.



Fig. 37. – Exemple CRF 37 (basse qualité)

Cet exemple est intéressant car il montre un désaccord entre le résultat visuel qui semble avoir permis d'éviter un bruit de compression important et gênant et le résultat de la métrique UVQ qui voit ici une baisse de qualité. Le reste des métriques s'accordent pour un gain.

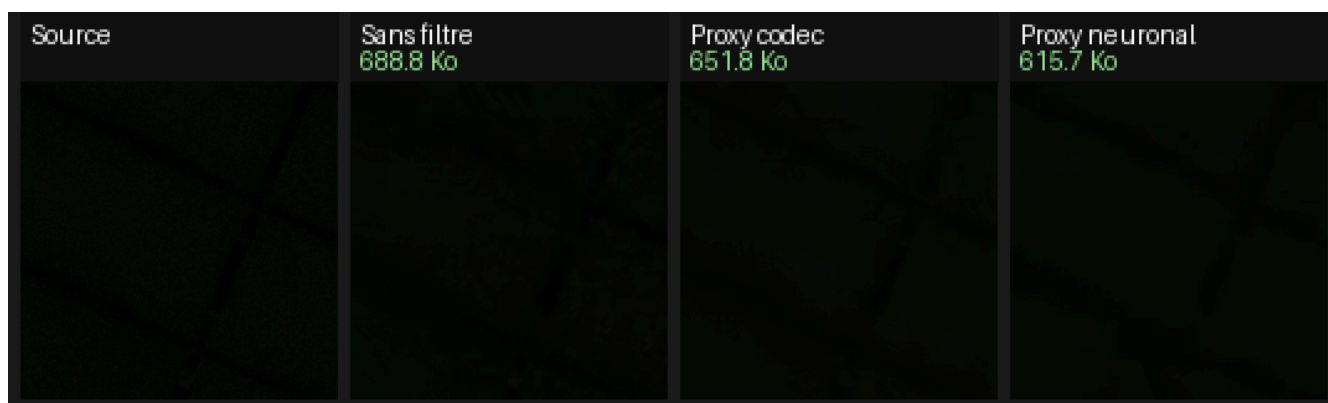


Fig. 38. – Exemple CRF 27 (qualité moyenne)

Cet exemple est aussi parlant (mais sombre, donc parfois difficile à observer), la métrique UVQ semble en désaccord avec le gain visuel, le reste des métriques s'accordent pour un gain minime, seul LPIPS donne un score bien plus important.



Fig. 39. – Exemple CRF 27 (qualité moyenne)

On voit ici que le filtre « Neuro » apporte un flou important, cependant la métrique VMAF y voit un gain, ce qui montre une incohérence avec le résultat visuel.

Ces exemples montrent qu'il existe parfois des contradictions dans les résultats, visuellement la qualité semble avoir été mieux conservée et le poids de la vidéo diminué, mais certaines métriques ne s'accordent pas sur ce résultat. Ce qui semble normal, la différence étant faible et ces mesures étant imparfaites, il se peut qu'elles se trompent parfois. On peut donc difficilement conclure à partir des résultats actuels. Les quelques exemples montrent tout de même que le filtre peut avoir un intérêt, et le fait qu'un bruit de compression soit évité montre bien que les outils fonctionnent. Cela indique que l'image a été simplifiée, ce qui a permis que l'étape de quantification, qui simplifie l'information, n'ait pas eu un effet trop délétère, ce qui est bien plus visible à basse qualité.

6.7. Limites et perspectives

Les résultats ne permettent pas d'établir une optimisation claire, visuellement certains exemples montrent quelques modifications qui semblent pertinentes quand on regarde les détails. Au regard des exemples visuels et résultats des métriques, on peut donc penser qu'à basse qualité le filtre a un intérêt pour conserver certaines textures ou limiter le bruit de compression. Nous sommes encore à un stade précoce du projet et ne pas avoir de résultats clairs semble normal, l'objectif ici était aussi de mettre en avant les différences entre les outils que l'on a mis en place et de voir comment ils se comportent.

Par la suite il faudra alors voir s'il est possible d'utiliser d'autres mesures afin de guider au mieux l'apprentissage car il semble assez clair, que le choix simple utilisé ici pour guider l'apprentissage n'est pas suffisant pour obtenir un gain net. Les deux options offrent tout de même des possibilités différentes. Le proxy simplifié pourra largement être modifié afin d'obtenir des performances plus intéressantes en repliquant par exemple des méthodes plus avancées, mais les scores obtenus pour sa fidélité d'image montrent tout de même que l'implémentation et les choix sont valides. La version « Neuro » semble détruire plus facilement les détails mais donc aussi réduire le poids de manière

plus importante, il serait possible de prendre ce résultat en compte et d'adapter certains critères pour éviter cela. Sa fidélité de reconstruction d'image montre que ce choix reste aussi intéressant. Enfin les différences liées aux deux options retenues (« Arrondi » et « Bruit ») ne sont pas significatives, il est donc difficile d'établir quel choix est le meilleur.

7. Conclusion

Ce projet de fin d'études s'attaque à une question concrète : peut-on, à l'aide de l'IA, optimiser la compression vidéo en amont d'un codec existant comme **H.265** ? Le cœur du travail a consisté à lever le principal verrou technique, l'impossibilité d'apprendre directement à travers un codec classique, en étudiant, adaptant et évaluant différentes approches de **proxy**. Deux voies ont été explorées : un proxy par codage neuronal et un codec simplifié différentiable, chacune avec ses forces et ses limites théoriques, le but était aussi d'évaluer leur pertinence sur un cas d'usage concret.

Les résultats ont montré que ces outils sont de bons imitateurs de la cible **H.265**, en particulier pour reproduire des images de bonne qualité.

Face aux métriques, ils font apparaître de légères optimisations à basse qualité, mais les écarts restent faibles, ce qui tient aussi à la difficulté de la tâche. Une limite doit d'ailleurs être posée. Les métriques actuelles ne sont pas assurément fiables pour notre cas d'usage, des images transformées par IA. On remarque cependant, sur plusieurs exemples visuels, que les modifications permettent parfois de préserver certaines textures ou de limiter le bruit de compression, ce qui montre qu'une optimisation de ce type semble possible avec ces outils. Il reste à en amplifier les effets pour obtenir des gains plus nets.

Ce travail s'accompagne d'un constat important : la qualité d'une optimisation dépend de nombreux éléments, le proxy, la **métrique** qui guide l'apprentissage, mais aussi les données utilisées ce qui rend le champ des possibilités immense.

Au-delà de l'aspect technique, ce rapport aura cherché à répondre à trois questions transversales. Sur le plan économique, on retiendra que les enjeux de la **VOD**, bande passante, stockage, énergie, rendent toute optimisation en amont directement profitable aux acteurs du secteur, et donc à notre cellule dans une logique de partenariat et donc de projets futurs. Sur le plan organisationnel, le projet illustre comment une petite structure, à l'interface d'un laboratoire et d'une entreprise, s'organise autour de réunions régulières et de ressources mutualisées et une adaptation au quotidien pour mener à bien ce projet. Sur le plan humain, enfin, il montre qu'une équipe jeune compense un certain manque d'expérience par une réelle capacité d'adaptation, une veille constante et un partage de connaissances au quotidien, autant d'atouts pour aborder des sujets de pointe.

Ce PFE ne constitue qu'une étape du projet, les outils mis en place ont vocation à être réutilisés, améliorés et adaptés. La validation finale, par de vrais utilisateurs, viendra confirmer la pertinence des optimisations une fois celles-ci suffisamment mûres.

8. Bilan personnel

Ce projet aura été, avant tout, une période d'apprentissage dense. J'y ai découvert un grand nombre de concepts, dans des domaines à la fois riches et exigeants, la compression vidéo comme le deep learning, et cette montée en compétence progressive constitue l'un des aspects les plus enrichissants de ces mois de travail.

Ce cheminement ne s'est pas fait sans difficultés. J'ai parfois eu du mal à me contenter de solutions simples, la recherche de résultats plus aboutis m'a conduit, à certains moments, vers des pistes trop ambitieuses, sources de frustration lorsqu'elles n'aboutissaient pas. J'ai appris, au fil du projet, à accepter la difficulté réelle de la tâche et à revenir vers des approches plus mesurées mais solides, quitte à progresser plus lentement. C'est sans doute l'un des enseignements les plus utiles de ce PFE, au-delà de son seul contenu technique.

Je dois aussi reconnaître que rester pleinement mobilisé sur des sujets aussi pointus, et sur une durée longue, n'est pas toujours évident. La complexité du domaine demande un effort de concentration soutenu, c'est ce qui rend l'organisation et la discipline de travail d'autant plus importantes.

Avec du recul, je pense qu'il aurait été plus sûr de rester sur une seule solution pour débiter, ce qui aurait facilité les différents tests, mais ce choix aura permis d'avoir plus d'outils pour la suite du projet.

J'ai par ailleurs conscience que les résultats obtenus restent, à ce stade, encore peu aboutis. Ce constat n'est pas un échec, il correspond aussi à la période du projet, qui est encore jeune, et il ouvre à des perspectives pour la suite. Ce PFE constitue donc un point d'étape important. Il aura permis de mettre au clair les idées, de développer les premiers outils et de dégager les directions qui guideront la poursuite du projet. C'est un travail de clarification et d'apprentissage dont la valeur dépasse celle des seuls résultats chiffrés.

Glossaire

AV1 Codec vidéo libre et ouvert, développé par l'Alliance for Open Media, successeur de VP9. Plus efficace que **H.265** à qualité égale, mais bien plus coûteux en calcul, ce qui limite son décodage sur le matériel ancien.

BD-rate Bjøntegaard Delta rate. Mesure de référence pour comparer deux codeurs : elle exprime l'économie moyenne de débit à qualité égale. Plus elle est négative, plus l'économie est importante.

CABAC Context-Adaptive Binary Arithmetic Coding. Méthode de codage entropique utilisée par **H.265**, combinant deux optimisations : le choix de probabilités selon le contexte (les coefficients voisins) et leur mise à jour adaptative au fil du codage.

CDN Content Delivery Network. Réseau de diffusion de contenu. Serveurs répartis mondialement qui stockent des copies de la vidéo pour la diffuser depuis le serveur le plus proche de l'utilisateur, réduisant la latence et les coûts.

Chrominance Composantes de couleur. Composantes d'une image portant l'information de couleur (chrominance bleue et chrominance rouge). L'œil y étant moins sensible, elles sont fortement sous-échantillonnées lors de la compression.

Codec Coder-decoder. Outil chargé de compresser la vidéo à l'encodage (côté fournisseur) et de la reconstituer au décodage (côté utilisateur).

Codec neuronal Codec dont les étapes de compression et de décompression sont réalisées par des réseaux de neurones. Entièrement différentiable/optimisable, il peut servir de **proxy** au sein d'un apprentissage, contrairement à un codec classique.

CRF / QP Constant Rate Factor / Quantization Parameter. Paramètres réglant l'intensité de la compression : plus leur valeur est élevée, plus la quantification est forte, donc plus la vidéo est légère mais dégradée. Le QP fixe un niveau de quantification rigide, ce qui aura un impact différent en fonction des zones de l'image. Le CRF vise une qualité constante, en essayant donc d'adapter la quantification à la spécificité de la zone.

CVVDP ColorVideoVDP. Métrique de qualité perceptuelle, modélisant la vision humaine (sensibilité au contraste, à la couleur et au mouvement) pour prédire la qualité perçue d'une vidéo. Contrairement à des mesures plus simples, elle traite les dimensions spatiale (détails dans l'image), temporelle (évolution entre les images) et chromatique (couleur), ce qui la rend particulièrement adaptée à l'évaluation de contenus dégradés ou modifiés.

DCT Discrete Cosine Transform. Transformée en cosinus discrète. Transformation appliquée aux blocs de l'image qui réorganise l'information par fréquences, séparant les zones lisses (basses fréquences) des zones de détail (hautes fréquences).

DISTS Deep Image Structure and Texture Similarity. Métrique de qualité perceptuelle s'appuyant sur un réseau de neurones, calibrée sur des jugements humains. Sa particularité est sa tolérance aux différences de texture, ce qui en fait un guide adapté à l'optimisation visée par ce projet.

Filtre Dans ce projet, réseau de neurones appliqué en amont de la compression (prétraitement) pour modifier l'image afin de la rendre plus facile à compresser, sans dégrader la qualité perçue.

Forward / Backward Les deux passes du graphe de calcul d'un réseau de neurones. Le *forward* (aller) calcule le résultat à partir des entrées, le *backward* (retour) remonte le graphe pour calculer les gradients qui permettent d'ajuster les paramètres.

FPS Frames Per Second. Images par seconde. Nombre d'images affichées chaque seconde dans une vidéo (couramment 30 ou 60). Plus il est élevé, plus la redondance temporelle entre images consécutives est forte.

GOP Group Of Pictures. Groupe d'images. Ensemble d'images consécutives codées ensemble, organisé autour d'une image de référence (intra) dont dépendent les images suivantes (prédites). Limiter sa taille évite de s'appuyer sur une image d'une autre scène.

H.264 Advanced Video Coding (AVC). Norme de compression vidéo créée en 2003. Bien qu'ancien, il reste le codec le plus utilisé aujourd'hui, grâce à son support matériel quasi universel.

HEVC / H.265 High Efficiency Video Coding. Norme de compression vidéo, successeur de **H.264**, offrant une efficacité accrue à qualité égale. C'est le codec ciblé par ce projet, retenu pour son support matériel répandu.

LPIPS Learned Perceptual Image Patch Similarity. Métrique de qualité perceptuelle fondée sur les activations d'un réseau de neurones, cherchant à corrélérer avec le jugement humain de similarité entre deux images.

Luminance Intensité lumineuse. Composante d'une image représentant son intensité lumineuse (les niveaux de gris). L'œil humain y est très sensible. C'est sur cette composante que se concentre le filtre développé dans ce projet.

Métrique Évaluation de qualité. Mesure visant à évaluer la qualité d'une vidéo, le plus souvent en cherchant à prédire le jugement qu'en porterait un utilisateur humain.

MOS Mean Opinion Score. Note d'opinion moyenne. Moyenne des notes de qualité attribuées par un panel d'utilisateurs à un contenu vidéo. C'est la référence « humaine » à laquelle on compare les métriques pour évaluer leur fiabilité.

Proxy Modèle de substitution. Dans ce projet, modèle de substitution différentiable reproduisant le comportement d'un codec réel, afin de pouvoir guider l'apprentissage d'un réseau de neurones là où le codec d'origine ne le permettrait pas.

PSNR Peak Signal-to-Noise Ratio. Mesure de fidélité comparant pixel à pixel l'image dégradée à la source. Simple à calculer, mais corrèle mal avec la perception humaine.

Quantification Réduction de précision. Étape de la compression qui réduit la précision des coefficients issus de la **DCT** (division puis arrondi). C'est l'étape où l'information est volontairement perdue, principalement sur les hautes fréquences, pour alléger le poids de la vidéo.

RGB Red, Green, Blue. Espace de représentation des couleurs où chaque pixel est décrit par trois valeurs (rouge, vert, bleu). Ces canaux sont fortement corrélés, ce qui le rend peu efficace pour la compression.

SSIM Structural Similarity Index Measure. Métrique de qualité comparant la similarité structurale entre deux images (luminance, contraste, structure). Plus proche de la perception humaine que le **PSNR**.

STE Straight Through Estimator. Technique permettant de faire traverser une opération non dérivable. On exécute l'opération réelle à l'aller (forward) et on lui substitue une approximation dérivable au retour (backward). C'est ce qui rend possible l'usage d'un codec au cœur de l'apprentissage.

SVR Support Vector Regression. Modèle d'apprentissage automatique « frugal » (peu coûteux par rapport à un réseau de neurones), utilisé notamment par la métrique **VMAF** pour relier les caractéristiques extraites d'une vidéo à un score de qualité.

VMAF Video Multimethod Assessment Fusion. Métrique de qualité développée par Netflix, devenue un standard de l'industrie. Elle combine plusieurs indicateurs extraits de l'image et du mouvement via un modèle appris (**SVR**) pour prédire la note qu'un utilisateur moyen donnerait. Elle ne traite que la luminance.

VOD Video On Demand. Vidéo à la demande. Mode de diffusion où chaque utilisateur déclenche un flux vidéo dédié au moment qu'il choisit, par opposition à une diffusion unique captée simultanément par tous comme la TNT.

YUV Luminance / Chrominance. Espace de représentation séparant l'intensité lumineuse (luminance) des informations de couleur (chrominances). Mieux adapté à la compression que le **RGB**, car il permet de réduire la couleur sans trop affecter la perception.

Références bibliographiques

- [1] R. Coffie, « Introduction to Live Streaming Architecture ». [En ligne]. Disponible sur: https://dev.to/robert_coffie/introduction-to-live-streaming-architecture-25i3
- [2] AppLogic Networks, « The Global Internet Phenomena Report ». février 2025.
- [3] Global Market Insights, « SVOD Market Size, Share and Growth Forecast 2024–2032 ». 2024.
- [4] Arcom et ADEME, « Présentation de l'étude sur l'impact environnemental des usages audiovisuels », technical report, 2024. Consulté le: 25 juin 2026. [En ligne]. Disponible sur: https://www.ademe.fr/wp-content/uploads/2024/12/Arcom-presentation-etude-impact-environnemental-des-usages-audiovisuels_0.pdf
- [5] L. Moreira, « Digital Video Introduction: A hands-on introduction to video technology ». [En ligne]. Disponible sur: https://github.com/leandromoreira/digital_video_introduction#basic-terminology
- [6] Bitmovin, « 7th Annual Video Developer Report 2023/2024 », Rapport industriel, 2023. [En ligne]. Disponible sur: <https://bitmovin.com/downloads/bitmovin-7th-video-developer-report-2023-2024.pdf>
- [7] Bitmovin, « 8th Annual Video Developer Report 2024/2025 », Rapport industriel, 2024. [En ligne]. Disponible sur: <https://bitmovin.com/downloads/bitmovin-8th-video-developer-report-2024-2025.pdf>
- [8] A. C. Edmondson, *Teaming: How Organizations Learn, Innovate, and Compete in the Knowledge Economy*. San Francisco: Jossey-Bass, 2012.
- [9] M. G. Morris et V. Venkatesh, « Age Differences in Technology Adoption Decisions: Implications for a Changing Work Force », *Personnel Psychology*, vol. 53, n° 2, p. 375-403, 2000.
- [10] Stack Overflow, « How to get the low, middle and high frequency components of an image ». 2015.
- [11] Wikimedia Commons, « CCD.png — décomposition d'une image en composantes RGB et YCbCr ».
- [12] Contributeurs de Wikipédia, « Chroma subsampling — Wikipedia, The Free Encyclopedia ». [En ligne]. Disponible sur: https://en.wikipedia.org/wiki/Chroma_subsampling
- [13] M. U. K. Khan, A. Chadha, M. A. Anam, et Y. Andreopoulos, « Perceptual Video Compression with Neural Wrapping », in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, p. 17743-17754.

- [14] A. Chadha et Y. Andreopoulos, « Deep Perceptual Preprocessing for Video Coding », in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, p. 14852-14861.
- [15] B. Isik, O. G. Guleryuz, D. Tang, J. Taylor, et P. A. Chou, « Sandwiched Video Compression: Efficiently Extending the Reach of Standard Codecs with Neural Wrappers », in *IEEE International Conference on Image Processing (ICIP)*, 2023.
- [16] J. Li, B. Li, et Y. Lu, « Neural Video Compression with Feature Modulation », in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, p. 26099-26108.

Annexe A Étude sur les plus gros challenges du secteur VOD/ Streaming

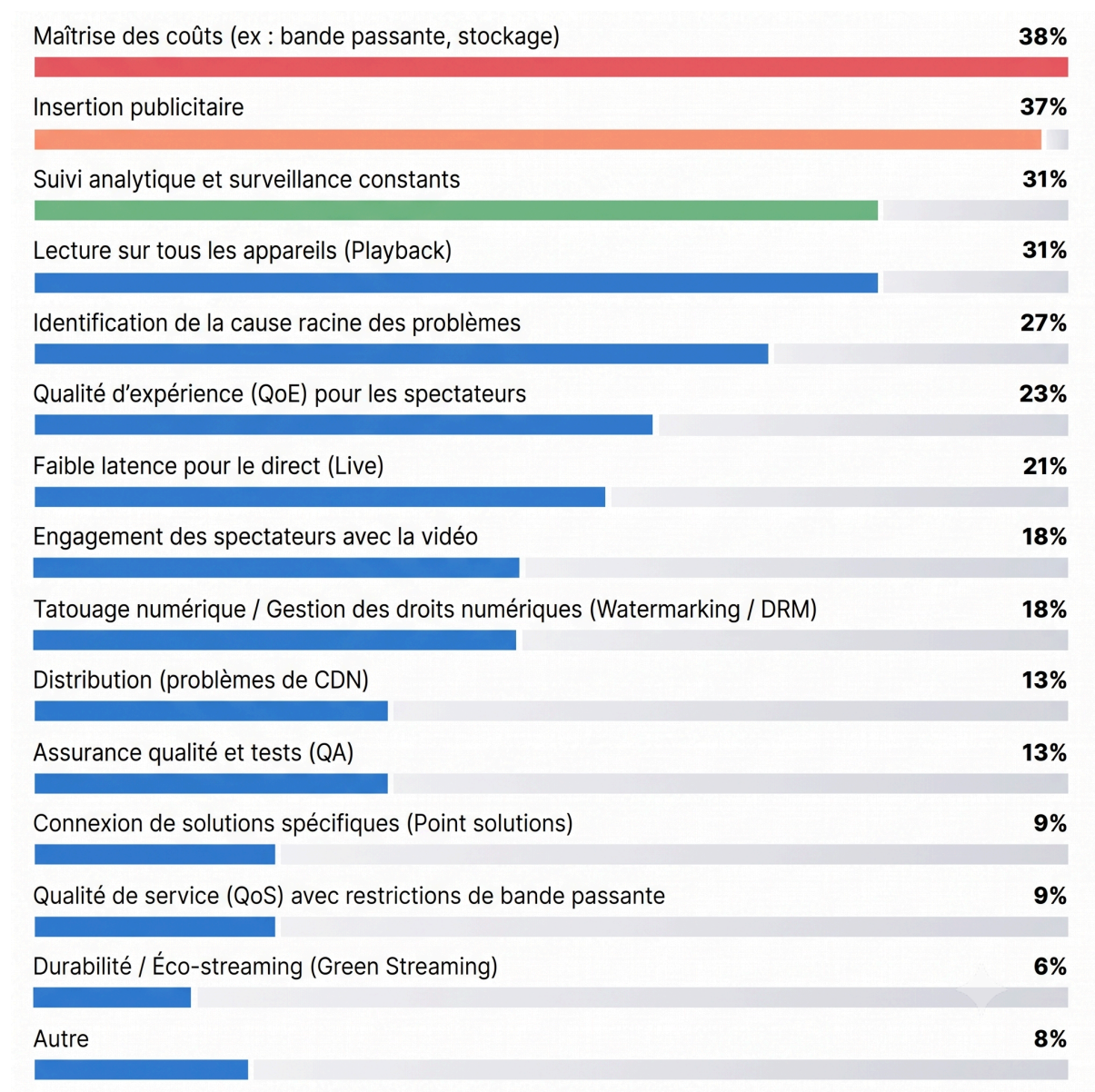


Fig. 40. – Illustration des challenges principaux face à un panel d'entreprises du secteur Streaming/
VOD pour l'année 2024-2025 [7]

Annexe B Architecture du filtre neuronal

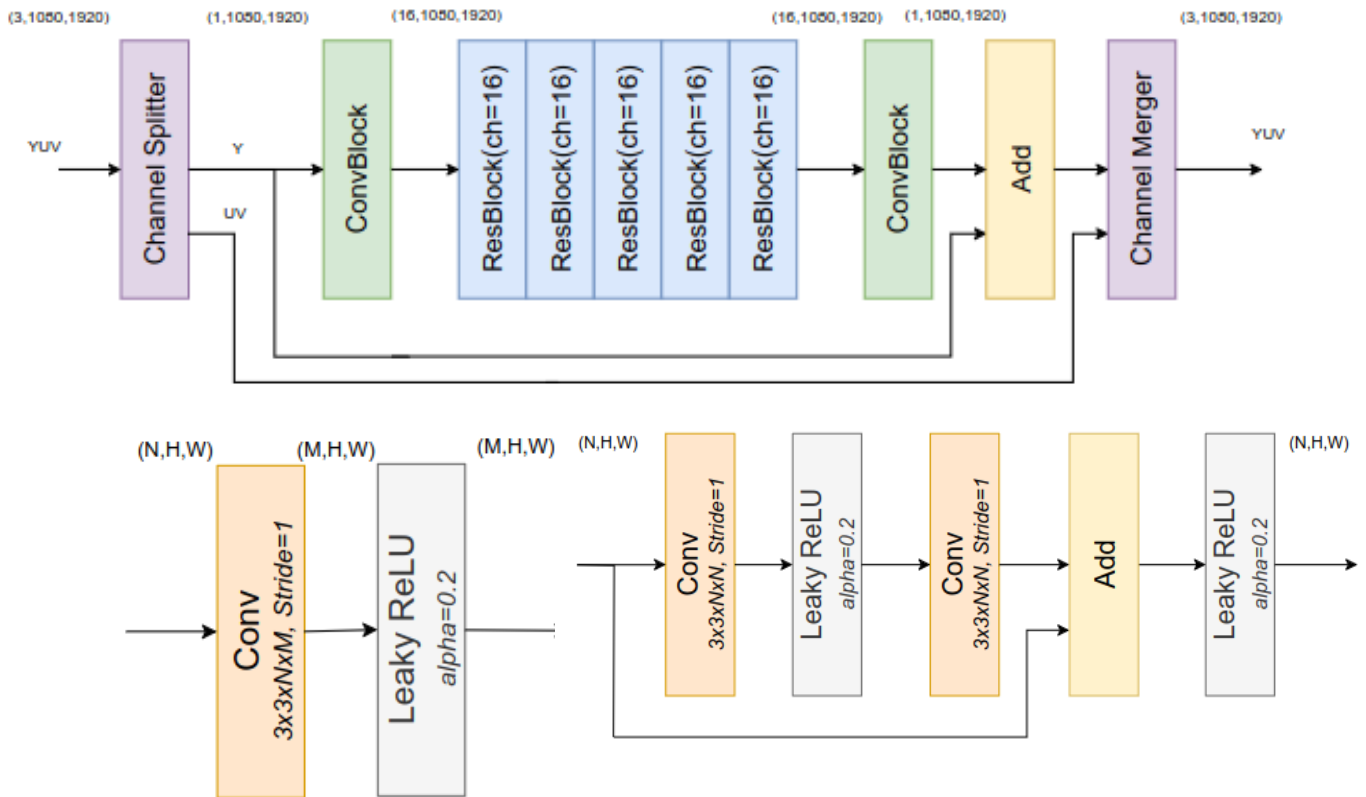


Fig. 41. – Architecture du réseau et ses briques de base : architecture globale (haut), en bas, convblock (gauche) et resblock (droite). [13]

Remerciements

Je remercie tout d'abord les différentes parties qui m'ont permis de réaliser ce projet de fin d'études : Capacités, Polytech Nantes ainsi que l'ITII.

Je remercie aussi l'équipe IPI qui n'est pas directement liée à mon contrat mais fait partie de mon quotidien et m'offre un environnement de travail agréable ainsi que la proximité à différentes expertises, ce qui m'a permis d'évoluer durant ces trois années d'alternance.

Je remercie aussi mon équipe tutorale, notamment Bruno Theillac, mon référent apprentissage, pour ses nombreux conseils, Matthieu Perreira Da Silva, mon tuteur pédagogique, pour ses conseils et ses retours notamment sur la vulgarisation technique et les contraintes du PFE. Enfin mon tuteur industriel Pierre Lebreton, pour sa confiance durant ce projet, ses conseils et sa disponibilité.

Résumé

Français

Ce projet de fin d'études explore une question du secteur de la vidéo à la demande : peut-on, grâce à l'intelligence artificielle, optimiser la compression vidéo en amont d'un codec existant comme H.265 ? L'idée est d'appliquer un filtre neuronal de prétraitement qui rend l'image plus facile à compresser, réduisant le poids du fichier final à qualité perçue équivalente. Le principal verrou est technique : les outils de compression classiques ne sont pas optimisables et s'intègrent donc mal dans l'apprentissage d'un réseau de neurones. Le travail a consisté à étudier, adapter et évaluer deux approches de proxy contournant cette limite : un proxy par codage neuronal, entraîné à imiter H.265, et un proxy par codec simplifié, reproduisant les briques essentielles de la compression sous une forme différentiable. Une attention particulière a été portée au choix des métriques de qualité, pour guider l'apprentissage comme pour évaluer les résultats sans biais. Au-delà de la technique, le rapport examine le projet sous les angles économique, organisationnel et humain.

Mots-clés : compression vidéo, H.265, apprentissage profond, prétraitement, proxy de codec, qualité perçue, métrique.

Anglais

This final-year project investigates a concrete challenge in the video-on-demand sector: can artificial intelligence be used to optimise video compression upstream of an existing codec such as H.265, without modifying end-user devices? The idea is to apply a neural pre-processing filter that makes each frame easier to compress, reducing the final file size at equivalent perceived quality. The main obstacle is technical: conventional codecs are not differentiable and therefore integrate poorly into the training of a neural network. The work consisted in studying, adapting and evaluating two proxy approaches to overcome this limitation: a neural-coding proxy, trained to mimic H.265, and a simplified-codec proxy, reproducing the essential building blocks of compression in a differentiable form. Particular attention was paid to the choice of quality metrics, both to guide learning and to evaluate results without bias. Beyond the technical aspect, this report also examines the project from economic, organisational and human perspectives.

Keywords : video compression, H.265, deep learning, pre-processing, codec proxy, perceived quality, metric.